```
// ==============================
// File: assets/scripts/AnalyticsManager.ts
// ==============================
import { _decorator, Component, sys } from 'cc';
import { PlayerDataManager } from './PlayerDataManager';
const { ccclass } = _decorator;

/**
 * 玩家操作类型枚举
 */
export enum OperationType {
    BUILD_WATCHTOWER = 'build_watchtower',          // 建造哨塔
    BUILD_WAR_ANCIENT_TREE = 'build_war_ancient_tree', // 建造战争古树
    BUILD_HUNTER_HALL = 'build_hunter_hall',        // 建造猎手大厅
    BUILD_SWORDSMAN_HALL = 'build_swordsman_hall',   // 建造剑士小屋
    BUILD_CHURCH = 'build_church',                  // 建造教堂
    BUILD_MOON_WELL = 'build_moon_well',            // 建造月亮井
    BUILD_STONE_WALL = 'build_stone_wall',          // 建造石墙
    BUILD_TREE = 'build_tree',                      // 建造树木
    BUILD_THUNDER_TOWER = 'build_thunder_tower',     // 建造雷塔
    BUILD_ICE_TOWER = 'build_ice_tower',            // 建造冰塔
    TRAIN_ARROWER = 'train_arrower',                // 训练弓箭手
    TRAIN_HUNTER = 'train_hunter',                  // 训练女猎手
    TRAIN_SWORDSMAN = 'train_swordsman',            // 训练剑士
    TRAIN_PRIEST = 'train_priest',                  // 训练牧师
    TRAIN_WISP = 'train_wisp',                      // 训练小精灵
    UPGRADE_UNIT = 'upgrade_unit',                  // 升级单位
    RECYCLE_UNIT = 'recycle_unit',                  // 回收单位
    UPGRADE_BUILDING = 'upgrade_building',          // 升级建筑
    RECYCLE_BUILDING = 'recycle_building',          // 回收建筑
    SELECT_BUFF_CARD = 'select_buff_card',          // 选择增益卡片
    USE_TALENT_POINT = 'use_talent_point',          // 使用天赋点
}

/**
 * 单次操作记录
 */
export interface OperationRecord {
    type: OperationType;          // 操作类型
    timestamp: number;            // 操作时间戳（毫秒）
    gameTime: number;             // 游戏内时间（秒）
    details?: any;                // 操作详情（可选）
}

/**
 * 游戏埋点数据
 */
export interface AnalyticsData {
    playerId: string;             // 玩家 ID
    level: number;                // 关卡数
    operations: OperationRecord[]; // 操作序列
    result: 'success' | 'fail';    // 游戏结果
    endTime: number;              // 结束时间戳
    defendTime: number;           // 已防御时间（秒）
    currentWave: number;          // 当前波次
    finalGold: number;            // 最终金币数
    finalPopulation: number;       // 最终人口数
    killCount: number;            // 击杀数
    unitLevels?: Record<string, number>; // 各单位强化等级（unitId -> level）
}
```

```
/**
 * 游戏埋点管理器
 * 负责记录玩家操作并在游戏结束时上报数据
 */
@ccclass('AnalyticsManager')
export class AnalyticsManager extends Component {
    private static instance: AnalyticsManager | null = null;

    private playerId: string = '';                  // 玩家 ID
    private currentLevel: number = 1;               // 当前关卡
    private operations: OperationRecord[] = [];      // 操作记录列表
    private gameStartTime: number = 0;              // 游戏开始时间
    private isRecording: boolean = false;            // 是否正在记录
    private playerDataManager: PlayerDataManager | null = null; // 玩家数据管理器（用于获取单位强化信息）

    // 服务器配置
    private readonly SERVER_URL = 'https://www.egoistcookie.top/api/analytics/report';

    /**
     * 获取单例实例
     */
    public static getInstance(): AnalyticsManager {
        if (!AnalyticsManager.instance) {
            console.warn('[AnalyticsManager] 实例未初始化');
        }
        return AnalyticsManager.instance!;
    }

    onLoad() {
        if (AnalyticsManager.instance) {
            this.destroy();
            return;
        }
        AnalyticsManager.instance = this;

        // 初始化玩家 ID（从本地存储获取或生成新的）
        this.initPlayerId();

        // 初始化玩家数据管理器（用于在埋点中附带单位强化信息）
        this.playerDataManager = PlayerDataManager.getInstance();
        this.playerDataManager.loadData().catch((err) => {
            console.warn('[AnalyticsManager] 加载玩家数据失败，单位强化信息将不会包含在埋点中:', err);
        });

        console.log('[AnalyticsManager] 埋点管理器初始化完成');
    }

    onDestroy() {
        if (AnalyticsManager.instance === this) {
            AnalyticsManager.instance = null;
        }
    }

    /**
     * 初始化玩家 ID
     */
    private initPlayerId() {
        const savedId = sys.localStorage.getItem('player_id');
        if (savedId) {
            this.playerId = savedId;
        } else {
```

```
            // 生成新的玩家 ID（时间戳 + 随机数）
            this.playerId = `player_${Date.now()}_${Math.floor(Math.random() *
10000)}`;
            sys.localStorage.setItem('player_id', this.playerId);
        }
        console.log('[AnalyticsManager] 玩家 ID:', this.playerId);
    }

    /**
     * 开始记录游戏
     */
    public startRecording(level: number) {
        this.currentLevel = level;
        this.operations = [];
        this.gameStartTime = Date.now();
        this.isRecording = true;
        console.log(`[AnalyticsManager] 开始记录关卡 ${level}`);
    }

    /**
     * 停止记录
     */
    public stopRecording() {
        this.isRecording = false;
        console.log('[AnalyticsManager] 停止记录');
    }

    /**
     * 记录操作
     */
    public recordOperation(type: OperationType, gameTime: number, details?: any) {
        if (!this.isRecording) {
            return;
        }

        const record: OperationRecord = {
            type,
            timestamp: Date.now(),
            gameTime,
            details
        };

        this.operations.push(record);
        console.log(`[AnalyticsManager] 记录操作: ${type}, 游戏时间:
${gameTime.toFixed(1)}s`);
    }

    /**
     * 上报游戏数据
     */
    public async reportGameData(
        result: 'success' | 'fail',
        defendTime: number,
        currentWave: number,
        finalGold: number,
        finalPopulation: number,
        killCount: number
    ): Promise<boolean> {
        if (!this.isRecording && this.operations.length === 0) {
            console.warn('[AnalyticsManager] 没有可上报的数据');
            return false;
        }
```

```
        this.stopRecording();

        // 从玩家数据中获取所有单位的强化等级（例如 Arrower: 15, StoneWall: 44）
        let unitLevels: Record<string, number> = {};
        if (this.playerDataManager) {
            try {
                unitLevels = this.playerDataManager.getAllUnitLevels();
            } catch (e) {
                console.warn('[AnalyticsManager] 获取单位等级信息失败,将跳过该字段:', e);
            }
        }

        const analyticsData: AnalyticsData = {
            playerId: this.playerId,
            level: this.currentLevel,
            operations: this.operations,
            result,
            endTime: Date.now(),
            defendTime,
            currentWave,
            finalGold,
            finalPopulation,
            killCount,
            unitLevels
        };

        // 转换为 JSON 字符串
        const jsonData = JSON.stringify(analyticsData);

        console.log('[AnalyticsManager] 准备上报数据:', {
            playerId: this.playerId,
            level: this.currentLevel,
            result,
            operationCount: this.operations.length,
            defendTime,
            currentWave
        });

        // 发送到服务器
        return this.sendToServer(jsonData);
    }

    /**
     * 发送数据到服务器
     */
    private async sendToServer(jsonData: string): Promise<boolean> {
        try {
            // 使用 XMLHttpRequest 发送 POST 请求
            return new Promise<boolean>((resolve) => {
                const xhr = new XMLHttpRequest();
                xhr.timeout = 5000; // 5 秒超时

                xhr.onreadystatechange = () => {
                    if (xhr.readyState === 4) {
                        if (xhr.status === 200) {
                            console.log('[AnalyticsManager]  数 据 上 报 成 功 :',
xhr.responseText);
                            resolve(true);
                        } else {
                            console.error('[AnalyticsManager] 数据上报失败:', xhr.status,
xhr.statusText);
                            resolve(false);
                        }
```

```
                }
            };

            xhr.onerror = () => {
                console.error('[AnalyticsManager] 网络请求失败');
                resolve(false);
            };

            xhr.ontimeout = () => {
                console.error('[AnalyticsManager] 请求超时');
                resolve(false);
            };

            xhr.open('POST', this.SERVER_URL, true);
            xhr.setRequestHeader('Content-Type', 'application/json');
            xhr.send(jsonData);
        });
    } catch (error) {
        console.error('[AnalyticsManager] 发送数据异常:', error);
        return false;
    }
}

/**
 * 获取当前操作数量
 */
public getOperationCount(): number {
    return this.operations.length;
}

/**
 * 清空操作记录
 */
public clearOperations() {
    this.operations = [];
    console.log('[AnalyticsManager] 操作记录已清空');
}
}


// ==============================
// File: assets/scripts/Arrow.ts
// ==============================
import { _decorator, Component, Node, Vec3, Sprite, tween, find } from 'cc';
import { GameState } from './GameState';
const { ccclass, property } = _decorator;

@ccclass('Arrow')
export class Arrow extends Component {
    @property
    speed: number = 500; // 弓箭飞行速度（像素/秒）

    @property
    arcHeight: number = 50; // 抛物线弧度高度（像素）

    @property
    damage: number = 10; // 伤害值

    private targetNode: Node = null!;
    private startPos: Vec3 = new Vec3();
    private targetPos: Vec3 = new Vec3();
    private travelTime: number = 0;
    private elapsedTime: number = 0;
```

```
    private onHitCallback: ((damage: number) => void) | null = null;
    private isFlying: boolean = false;
    private lastPos: Vec3 = new Vec3();
    private gameManager: any = null; // GameManager 引用（使用 any 避免循环依赖）

    /**
     * 初始化弓箭
     * @param startPos 起始位置
     * @param targetNode 目标节点
     * @param damage 伤害值
     * @param onHit 命中回调函数
     */
    init(startPos: Vec3, targetNode: Node, damage: number, onHit?: (damage: number) =>
void) {
        this.startPos = startPos.clone();
        this.targetNode = targetNode;
        this.damage = damage;
        this.onHitCallback = onHit || null;

        // 设置初始位置
        this.node.setWorldPosition(this.startPos);

        // 计算目标位置
        if (this.targetNode && this.targetNode.isValid) {
            this.targetPos = this.targetNode.worldPosition.clone();
        } else {
            // 如果目标无效，使用起始位置前方
            this.targetPos = this.startPos.clone();
            this.targetPos.x += 200;
        }

        // 计算飞行时间（基于距离和速度）
        const distance = Vec3.distance(this.startPos, this.targetPos);
        if (distance < 0.1) {
            return;
        }

        this.travelTime = distance / this.speed;
        if (this.travelTime <= 0) {
            return;
        }

        this.elapsedTime = 0;
        this.isFlying = true;
        this.lastPos = this.startPos.clone();

        // 设置初始旋转角度（指向目标）
        const initialDirection = new Vec3();
        Vec3.subtract(initialDirection, this.targetPos, this.startPos);
        if (initialDirection.length() > 0.1) {
            const angle = Math.atan2(initialDirection.y, initialDirection.x) * 180 /
Math.PI;
            this.node.setRotationFromEuler(0, 0, angle);
        }

    }

    /**
     * 计算抛物线位置
     * @param ratio 进度比例 (0-1)
     */
    calculateParabolicPosition(ratio: number): Vec3 {
        const pos = new Vec3();
```

```
        // 线性插值计算基础位置
        Vec3.lerp(pos, this.startPos, this.targetPos, ratio);

        // 添加抛物线高度（使用二次函数模拟重力效果）
        const arcRatio = 4 * ratio * (1 - ratio); // 0 到 1 再到 0 的曲线
        pos.y += this.arcHeight * arcRatio;

        return pos;
    }


    /**
     * 命中目标
     */
    hitTarget() {
        if (!this.isFlying) {
            return; // 已经命中过了
        }

        // 检查目标是否是已死亡的兽人督军
        if (this.targetNode && this.targetNode.isValid) {
            const orcScript = this.targetNode.getComponent('OrcWarlord') as any;
            if (orcScript) {
                const isAlive = orcScript.isAlive && orcScript.isAlive();
                if (!isAlive) {
                    // 目标是已死亡的兽人督军，插在尸体上
                    const startPos = this.lastPos.clone();
                    const endPos = this.node.worldPosition.clone();
                    this.attachToCorpse(this.targetNode, startPos, endPos);
                    return;
                }
            }
        }

        this.isFlying = false;

        // 调用命中回调
        if (this.onHitCallback) {
            this.onHitCallback(this.damage);
        }

        // 立即销毁弓箭节点，避免在 timeScale=0 时 scheduleOnce 不执行
        if (this.node && this.node.isValid) {
            this.node.destroy();
        }
    }

    update(deltaTime: number) {
        if (!this.isFlying) {
            return;
        }

        // 检查游戏状态 - 如果 GameManager 不存在，尝试重新查找（使用字符串避免循环依赖）
        if (!this.gameManager) {
            const gmNode = find('GameManager') || find('Canvas/GameManager');
            if (gmNode) {
                this.gameManager = gmNode.getComponent('GameManager' as any);
            }
        }

        // 检查游戏状态，如果不是 Playing 状态，停止飞行
        if (this.gameManager && this.gameManager.getGameState) {
```

```
            const gameState = this.gameManager.getGameState();
            if (gameState !== GameState.Playing) {
                // 游戏已暂停或结束，停止飞行
                return;
            }
        }

        // 更新计时器
        this.elapsedTime += deltaTime;

        // 调试：每 0.5 秒输出一次位置信息
        if  (Math.floor(this.elapsedTime  *  2)  !==  Math.floor((this.elapsedTime  -
deltaTime) * 2)) {
            const                              currentPos                              =
this.calculateParabolicPosition(Math.min(this.elapsedTime / this.travelTime, 1));
        }

        // 如果目标已失效，提前销毁
        if (this.targetNode && (!this.targetNode.isValid || !this.targetNode.active))
{
            if (this.node && this.node.isValid) {
                this.node.destroy();
            }
            return;
        }

        // 更新目标位置（目标可能在移动）
        if (this.targetNode && this.targetNode.isValid) {
            this.targetPos = this.targetNode.worldPosition.clone();
        }

        // 计算当前进度
        const currentRatio = Math.min(this.elapsedTime / this.travelTime, 1);

        // 计算当前在抛物线上的位置
        const currentPos = this.calculateParabolicPosition(currentRatio);

        // 检查路径上是否有兽人督军尸体
        if (this.checkForOrcWarlordCorpse(this.lastPos, currentPos)) {
            return;
        }

        this.node.setWorldPosition(currentPos);

        // 更新旋转角度，使箭头始终指向飞行方向
        const direction = new Vec3();
        Vec3.subtract(direction, currentPos, this.lastPos);
        if (direction.length() > 0.1) {
            const angle = Math.atan2(direction.y, direction.x) * 180 / Math.PI;
            this.node.setRotationFromEuler(0, 0, angle);
        }
        this.lastPos = currentPos.clone();

        // 检查是否命中目标
        if (this.targetNode && this.targetNode.isValid) {
            const currentTargetPos = this.targetNode.worldPosition;
            const distance = Vec3.distance(currentPos, currentTargetPos);

            // 如果距离足够近，认为命中
            if (distance < 30) {
                this.hitTarget();
                return;
            }
```

```
        }

        // 如果飞行完成但未命中目标，销毁弓箭
        if (currentRatio >= 1) {
            if (this.node && this.node.isValid) {
                this.node.destroy();
            }
            return;
        }

        // 如果飞行时间过长（超过预期时间 2 倍），销毁弓箭（防止卡住）
        if (this.elapsedTime > this.travelTime * 2) {
            if (this.node && this.node.isValid) {
                this.node.destroy();
            }
        }
    }

    /**
     * 递归查找节点
     * @param node 起始节点
     * @param name 节点名称
     * @returns 找到的节点
     */
    findNodeRecursive(node: Node, name: string): Node | null {
        if (node.name === name) {
            return node;
        }
        for (const child of node.children) {
            const found = this.findNodeRecursive(child, name);
            if (found) {
                return found;
            }
        }
        return null;
    }

    /**
     * 检查路径上是否有兽人督军尸体
     * @param startPos 起始位置
     * @param endPos 结束位置
     * @returns 是否命中尸体
     */
    checkForOrcWarlordCorpse(startPos: Vec3, endPos: Vec3): boolean {
        // 查找 Enemies 容器，使用直接路径
        const enemiesNode = find('Canvas/Enemies');

        if (!enemiesNode) {
            return false;
        }

        const enemies = enemiesNode.children || [];

        for (const enemy of enemies) {
            if (!enemy || !enemy.isValid) {
                continue;
            }

            // 检查是否是兽人督军
            const orcScript = enemy.getComponent('OrcWarlord') as any;
            if (!orcScript) {
                continue;
            }
```

```
        // 检查是否已经死亡
        const isAlive = orcScript.isAlive && orcScript.isAlive();
        if (!isAlive && enemy.isValid) {
            // 检查整个飞行路径是否与尸体相交
            const corpsePos = enemy.worldPosition;
            const distance = this.getDistanceFromLine(startPos, endPos, corpsePos);

            // 如果距离小于 30 像素，认为命中尸体
            if (distance < 30) {
                // 命中尸体，插在尸体身上
                this.attachToCorpse(enemy, startPos, endPos);
                return true;
            }
        }
    }

    return false;
}

/**
 * 将箭矢插在尸体身上
 * @param corpseNode 尸体节点
 * @param startPos 起始位置
 * @param endPos 结束位置
 */
attachToCorpse(corpseNode: Node, startPos: Vec3, endPos: Vec3): void {
    if (!this.node || !this.node.isValid || !corpseNode || !corpseNode.isValid) {
        return;
    }

    // 计算命中点
    const corpsePos = corpseNode.worldPosition;
    const hitPos = this.calculateHitPoint(startPos, endPos, corpsePos);

    // 停止飞行状态
    this.isFlying = false;

    // 将箭矢设为尸体的子节点
    // 这样当尸体被销毁时，箭矢也会被自动销毁
    this.node.removeFromParent();
    corpseNode.addChild(this.node);

    // 设置箭矢位置（转换为局部位置）
    const localPos = corpseNode.worldPosition.clone();
    Vec3.subtract(localPos, hitPos, localPos);
    this.node.setPosition(localPos);

    // 调整箭矢旋转，使其指向飞行方向
    const direction = new Vec3();
    Vec3.subtract(direction, endPos, startPos);
    if (direction.length() > 0.1) {
        const angle = Math.atan2(direction.y, direction.x) * 180 / Math.PI;
        this.node.setRotationFromEuler(0, 0, angle);
    }

    // 命中尸体不触发伤害回调
    // 清除回调，防止后续调用
    this.onHitCallback = null;
}

/**
 * 计算命中点
```

```
     * @param startPos 起始位置
     * @param endPos 结束位置
     * @param corpsePos 尸体位置
     * @returns 命中点位置
     */
    calculateHitPoint(startPos: Vec3, endPos: Vec3, corpsePos: Vec3): Vec3 {
        // 计算线段方向
        const lineDir = Vec3.subtract(new Vec3(), endPos, startPos);
        const lineLengthSqr = Vec3.lengthSqr(lineDir);

        if (lineLengthSqr === 0) {
            return corpsePos.clone();
        }

        // 计算投影点
        const t = Math.max(0, Math.min(1, Vec3.dot(Vec3.subtract(new Vec3(), corpsePos,
startPos), lineDir) / lineLengthSqr));
        const projection = Vec3.add(new Vec3(), startPos, Vec3.multiplyScalar(new Vec3(),
lineDir, t));

        return projection;
    }

    /**
     * 计算点到线段的最短距离
     * @param lineStart 线段起点
     * @param lineEnd 线段终点
     * @param point 点
     * @returns 最短距离
     */
    getDistanceFromLine(lineStart: Vec3, lineEnd: Vec3, point: Vec3): number {
        const lineDir = Vec3.subtract(new Vec3(), lineEnd, lineStart);
        const lineLengthSqr = Vec3.lengthSqr(lineDir);

        if (lineLengthSqr === 0) {
            // 线段长度为0，直接返回点到起点的距离
            return Vec3.distance(point, lineStart);
        }

        const t = Math.max(0, Math.min(1, Vec3.dot(Vec3.subtract(new Vec3(), point,
lineStart), lineDir) / lineLengthSqr));
        const projection = Vec3.add(new Vec3(), lineStart, Vec3.multiplyScalar(new
Vec3(), lineDir, t));

        return Vec3.distance(point, projection);
    }
}



// ==============================
// File: assets/scripts/Arrow2.ts
// ==============================
import { _decorator, Component, Node, Vec3, find } from 'cc';
import { GameManager, GameState } from './GameManager';
const { ccclass, property } = _decorator;

/**
 * Arrow2: 3级弓箭手专用的穿透箭
 * - 直线弹道（无抛物线）
 * - 穿透第一个敌人后继续飞行100像素
 */
@ccclass('Arrow2')
```

```
export class Arrow2 extends Component {
    @property
    speed: number = 500; // 弓箭飞行速度（像素/秒）

    @property
    damage: number = 10; // 伤害值

    private startPos: Vec3 = new Vec3();
    private direction: Vec3 = new Vec3(); // 飞行方向（归一化）
    private travelDistance: number = 0; // 总飞行距离
    private traveledDistance: number = 0; // 已飞行距离
    private onHitCallback: ((damage: number, hitPos: Vec3, enemy: Node) => void) | null
= null;
    private isFlying: boolean = false;
    private gameManager: GameManager | null = null;
    private hasHitFirstTarget: boolean = false; // 是否已命中第一个目标
    private firstHitPos: Vec3 = new Vec3(); // 第一个命中点的位置
    private penetrationDistance: number = 100; // 穿透后继续飞行的距离（像素）
    private hitEnemies: Set<Node> = new Set(); // 已命中的敌人集合，避免重复伤害

    /**
     * 初始化穿透箭
     * @param startPos 起始位置
     * @param targetNode 目标节点（用于计算初始方向）
     * @param damage 伤害值
     * @param onHit 命中回调函数（参数：伤害值，命中位置，敌人节点）
     */
    init(startPos: Vec3, targetNode: Node, damage: number, onHit?: (damage: number,
hitPos: Vec3, enemy: Node) => void) {
        this.startPos = startPos.clone();
        this.damage = damage;
        this.onHitCallback = onHit || null;
        this.hasHitFirstTarget = false;
        this.traveledDistance = 0;
        this.hitEnemies.clear(); // 清空已命中敌人集合

        // 设置初始位置
        this.node.setWorldPosition(this.startPos);

        // 计算初始方向（从起始位置指向目标）
        let targetPos: Vec3;
        if (targetNode && targetNode.isValid) {
            targetPos = targetNode.worldPosition.clone();
        } else {
            // 如果目标无效，使用起始位置前方
            targetPos = this.startPos.clone();
            targetPos.x += 200;
        }

        // 计算方向向量
        Vec3.subtract(this.direction, targetPos, this.startPos);
        const distance = this.direction.length();

        if (distance < 0.1) {
            // 距离太近，无法初始化
            this.scheduleOnce(() => {
                if (this.node && this.node.isValid) {
                    this.node.destroy();
                }
            }, 0);
            return;
        }
```

```
        // 归一化方向
        this.direction.normalize();

        // 计算总飞行距离：到第一个目标的距离 + 穿透距离
        this.travelDistance = distance + this.penetrationDistance;

        // 设置初始旋转角度（指向目标）
        const angle = Math.atan2(this.direction.y, this.direction.x) * 180 / Math.PI;
        this.node.setRotationFromEuler(0, 0, angle);

        this.isFlying = true;
    }

    /**
     * 计算直线位置（无抛物线）
     * @param distance 已飞行距离
     */
    calculateStraightPosition(distance: number): Vec3 {
        const pos = new Vec3();
        Vec3.scaleAndAdd(pos, this.startPos, this.direction, distance);
        return pos;
    }

    /**
     * 命中目标（穿透）
     */
    hitTarget(hitPos: Vec3, enemy: Node) {
        if (!this.isFlying) {
            return; // 已经处理过了
        }

        // 检查是否已经命中过这个敌人
        if (this.hitEnemies.has(enemy)) {
            return; // 已经命中过，跳过
        }

        // 标记为已命中
        this.hitEnemies.add(enemy);

        // 调用命中回调
        if (this.onHitCallback) {
            this.onHitCallback(this.damage, hitPos, enemy);
        }

        // 如果还没命中第一个目标，记录命中位置，继续飞行
        if (!this.hasHitFirstTarget) {
            this.hasHitFirstTarget = true;
            this.firstHitPos = hitPos.clone();
            // 不停止飞行，继续穿透
        }
        // 如果已经命中第一个目标，继续飞行直到达到穿透距离
    }

    update(deltaTime: number) {
        if (!this.isFlying) {
            return;
        }

        // 检查游戏状态
        if (!this.gameManager) {
            this.gameManager = find('GameManager')?.getComponent(GameManager);
        }
```

```
        if (this.gameManager) {
            const gameState = this.gameManager.getGameState();
            if (gameState !== GameState.Playing) {
                return;
            }
        }

        // 计算移动距离
        const moveDistance = this.speed * deltaTime;
        this.traveledDistance += moveDistance;

        // 计算当前位置
        const currentPos = this.calculateStraightPosition(this.traveledDistance);
        this.node.setWorldPosition(currentPos);

        // 检查是否命中敌人
        this.checkHitEnemies(currentPos);

        // 检查是否达到最大飞行距离
        if (this.traveledDistance >= this.travelDistance) {
            // 飞行完成，销毁箭矢
            this.isFlying = false;
            this.scheduleOnce(() => {
                if (this.node && this.node.isValid) {
                    this.node.destroy();
                }
            }, 0.1);
            return;
        }

        // 如果飞行时间过长（超过预期时间 2 倍），销毁弓箭（防止卡住）
        const expectedTime = this.travelDistance / this.speed;
        if (this.traveledDistance > this.travelDistance * 2) {
            this.isFlying = false;
            this.scheduleOnce(() => {
                if (this.node && this.node.isValid) {
                    this.node.destroy();
                }
            }, 0.1);
        }
    }

    /**
     * 检查是否命中敌人
     */
    checkHitEnemies(currentPos: Vec3) {
        const enemiesNode = find('Canvas/Enemies');
        if (!enemiesNode) {
            return;
        }

        const enemies = enemiesNode.children || [];
        const hitRadius = 30; // 命中判定半径

        for (const enemy of enemies) {
            if (!enemy || !enemy.isValid || !enemy.active) {
                continue;
            }

            // 检查是否是敌人
            const enemyScript = enemy.getComponent('OrcWarlord') as any ||
                                enemy.getComponent('OrcWarrior') as any ||
                                enemy.getComponent('Enemy') as any ||
```

```
                            enemy.getComponent('TrollSpearman') as any;

            if (!enemyScript) {
                continue;
            }

            // 检查是否存活
            const isAlive = enemyScript.isAlive && enemyScript.isAlive();
            if (!isAlive) {
                continue;
            }

            // 计算距离
            const enemyPos = enemy.worldPosition;
            const distance = Vec3.distance(currentPos, enemyPos);

            // 如果距离足够近，认为命中
            if (distance < hitRadius) {
                // 命中敌人（穿透，不停止飞行）
                this.hitTarget(currentPos, enemy);
                // 继续飞行，不停止
                break; // 一次只命中一个敌人
            }
        }
    }
}



// ==============================
// File: assets/scripts/AudioManager.ts
// ==============================
import { _decorator, Component, AudioSource, AudioClip, Node, instantiate, Prefab, find,
director } from 'cc';
const { ccclass, property } = _decorator;

/**
 * 音频管理器，用于统一管理游戏中的音频播放
 * 使用单例模式，确保全局唯一实例
 * 使用 AudioSource 组件来播放音频
 */
@ccclass('AudioManager')
export class AudioManager extends Component {
    // 单例实例（仅在场景中创建一次，其余地方只获取）
    private static instance: AudioManager | null = null;

    // 背景音乐音量
    @property
    bgmVolume: number = 0.5;

    // 音效音量
    @property
    sfxVolume: number = 0.8;

    // 背景音乐 AudioSource 组件
    private bgmAudioSource: AudioSource = null!;

    // 音效节点池，用于播放多个音效
    private sfxNodes: Node[] = [];

    // 音效节点最大数量
    private readonly MAX_SFX_NODES: number = 10;
```

```
    /**
     * 获取单例实例（只在场景中查找，找不到则打印日志并返回 null）
     */
    public static get Instance(): AudioManager {
        // 如果已有有效实例，直接返回
        if (this.instance && this.instance.node && this.instance.node.isValid) {
            return this.instance;
        }

        // 尝试在当前场景中查找已有的 AudioManager 组件
        const scene = director.getScene();
        if (scene) {
            const stack: Node[] = [...scene.children];
            while (stack.length > 0) {
                const node = stack.pop()!;
                const comp = node.getComponent(AudioManager);
                if (comp && comp.node && comp.node.isValid) {
                   //console.info('[AudioManager]    Instance()    found    existing
AudioManager on node', node.name);
                    this.instance = comp;
                    return comp;
                }
                stack.push(...node.children);
            }
        }

        // 再尝试常见路径的节点名称
        const audioManagerNode = find('Canvas/AudioManager') || find('AudioManager');
        if (audioManagerNode) {
            const comp = audioManagerNode.getComponent(AudioManager);
            if (comp && comp.node && comp.node.isValid) {
               //console.info('[AudioManager] Instance() found AudioManager on node',
audioManagerNode.name);
                this.instance = comp;
                return comp;
            }
        }

        console.warn('[AudioManager] Instance() no AudioManager found in scene. Please
add one under Canvas.');
        return null as any;
    }

    protected onLoad(): void {
        // 确保只有一个 AudioManager 实例，销毁多余的
        if (AudioManager.instance && AudioManager.instance !== this) {
            console.warn('[AudioManager]    Duplicate    AudioManager    on    node',
this.node.name, ', destroying this one');
            this.node.destroy();
            return;
        }
        AudioManager.instance = this;

        // 如果当前节点还没有挂到场景上，则尝试挂到 Canvas 或场景根
        if (!this.node.scene) {
            const scene = director.getScene();
            const root = find('Canvas') || scene;
            if (root) {
                root.addChild(this.node);
            }
        }

        // 将 AudioManager 节点标记为常驻节点，防止在切换 / 重新加载场景时被销毁
```

```
        if (!director.isPersistRootNode(this.node)) {
            director.addPersistRootNode(this.node);
           //console.info('[AudioManager] onLoad() mark node as persist root');
        }

        // 初始化音频管理器
        this.initAudioManager();
    }

    /**
     * 确保背景音乐 AudioSource 已经创建并初始化
     */
    private ensureBgmAudioSource(): void {
        // 如果组件的 node 无效，直接放弃，避免报错
        if (!this.node || !this.node.isValid) {
            console.warn('[AudioManager] ensureBgmAudioSource() aborted because node is
invalid');
            return;
        }

        if         (!this.bgmAudioSource         ||         !this.bgmAudioSource.node
|| !this.bgmAudioSource.node.isValid) {
            this.bgmAudioSource     =     this.node.getComponent(AudioSource)     ||
this.node.addComponent(AudioSource);
        }

        if (!this.bgmAudioSource) {
            console.warn('[AudioManager] ensureBgmAudioSource() failed: bgmAudioSource
is still null');
            return;
        }
        // 确保关键属性正确
        this.bgmAudioSource.loop = true;
        this.bgmAudioSource.volume = this.bgmVolume;
    }

    /**
     * 初始化音频管理器
     */
    private initAudioManager(): void {
        // 创建并初始化背景音乐 AudioSource 组件
        this.ensureBgmAudioSource();

        // 初始化音效节点池
        this.initSFXNodes();
    }

    /**
     * 初始化音效节点池
     */
    private initSFXNodes(): void {
        for (let i = 0; i < this.MAX_SFX_NODES; i++) {
            const sfxNode = new Node(`SFX_${i}`);
            sfxNode.setParent(this.node);
            // 激活 SFX 节点，否则 AudioSource 组件无法正常工作
            sfxNode.active = true;
            const audioSource = sfxNode.addComponent(AudioSource);
            audioSource.loop = false;
            audioSource.volume = this.sfxVolume;
            this.sfxNodes.push(sfxNode);
        }
    }
```

```
    /**
     * 获取一个可用的音效节点
     * @returns 可用的音效节点
     */
    private getAvailableSFXNode(): Node | null {
        // 检查音效节点池是否已初始化
        if (this.sfxNodes.length === 0) {
            this.initSFXNodes();
        }

        // 查找未在播放的音效节点
        for (const node of this.sfxNodes) {
            const audioSource = node.getComponent(AudioSource);
            if (audioSource && !audioSource.playing) {
                return node;
            }
        }

        // 如果没有可用节点，返回第一个节点（覆盖最早的音效）
        const result = this.sfxNodes[0];
        return result;
    }

    /**
     * 播放背景音乐
     * @param clip 音频资源
     */
    public playBGM(clip: AudioClip): void {
        if (!clip) {
            return;
        }

        // 设置背景音乐并播放
        this.ensureBgmAudioSource();
        this.bgmAudioSource.clip = clip;
        this.bgmAudioSource.play();
    }

    /**
     * 停止背景音乐
     */
    public stopBGM(): void {
        this.ensureBgmAudioSource();
        if (this.bgmAudioSource) {
            this.bgmAudioSource.stop();
        }
    }

    /**
     * 暂停背景音乐
     */
    public pauseBGM(): void {
        this.ensureBgmAudioSource();
        if (this.bgmAudioSource) {
            this.bgmAudioSource.pause();
        }
    }

    /**
     * 恢复背景音乐
     */
    public resumeBGM(): void {
        this.ensureBgmAudioSource();
```

```
        if (this.bgmAudioSource) {
            this.bgmAudioSource.play();
        }
    }

    /**
     * 播放音效
     * @param clip 音频资源
     */
    public playSFX(clip: AudioClip): void {
        if (!clip) {
            return;
        }


        // 获取可用的音效节点
        const sfxNode = this.getAvailableSFXNode();
        if (!sfxNode) {
            return;
        }

        // 播放音效
        const audioSource = sfxNode.getComponent(AudioSource);
        if (audioSource) {
            // 使用 playOneShot 方法播放一次性音效，这是更可靠的方法
            audioSource.playOneShot(clip, this.sfxVolume);
        } else {
        }
    }

    /**
     * 停止所有音效
     */
    public stopAllSFX(): void {
        for (const node of this.sfxNodes) {
            // 添加 null 检查，确保节点有效
            if (node && node.isValid) {
                const audioSource = node.getComponent(AudioSource);
                if (audioSource) {
                    audioSource.stop();
                }
            }
        }
    }

    /**
     * 设置背景音乐音量
     * @param volume 音量值（0-1）
     */
    public setBGMVolume(volume: number): void {
        this.bgmVolume = Math.max(0, Math.min(1, volume));
        this.ensureBgmAudioSource();
        this.bgmAudioSource.volume = this.bgmVolume;
    }

    /**
     * 设置音效音量
     * @param volume 音量值（0-1）
     */
    public setSFXVolume(volume: number): void {
        this.sfxVolume = Math.max(0, Math.min(1, volume));

        // 更新所有音效节点的音量
```

```
        for (const node of this.sfxNodes) {
            const audioSource = node.getComponent(AudioSource);
            if (audioSource) {
                audioSource.volume = this.sfxVolume;
            }
        }
    }

    /**
     * 获取当前背景音乐音量
     * @returns 音量值（0-1）
     */
    public getBGMVolume(): number {
        return this.bgmVolume;
    }

    /**
     * 获取当前音效音量
     * @returns 音量值（0-1）
     */
    public getSFXVolume(): number {
        return this.sfxVolume;
    }

    protected onDestroy(): void {
        // 清理资源
        if (AudioManager.instance === this) {
            AudioManager.instance = null!;

            // 停止所有音频，添加 null 检查
            if (this.bgmAudioSource) {
                this.bgmAudioSource.stop();
            }
            this.stopAllSFX();
        }
    }
}


// ==============================
// File: assets/scripts/Boomerang.ts
// ==============================
import { _decorator, Component, Node, Vec3, Sprite, tween, find } from 'cc';
import { GameManager, GameState } from './GameManager';
const { ccclass, property } = _decorator;

@ccclass('Boomerang')
export class Boomerang extends Component {
    @property
    speed: number = 300; // 回旋镖飞行速度（像素/秒）

    @property
    arcHeight: number = 50; // 抛物线弧度高度（像素）

    @property
    damage: number = 10; // 初始伤害值

    @property
    maxBounces: number = 3; // 最大弹射次数

    @property
    bounceDamageMultiplier: number = 0.5; // 弹射伤害倍率
```

```
    @property
    bounceRange: number = 100; // 弹射距离阈值（像素）

    private targetNode: Node = null!;
    private startPos: Vec3 = new Vec3();
    private targetPos: Vec3 = new Vec3();
    private travelTime: number = 0;
    private elapsedTime: number = 0;
    private onHitCallback: ((damage: number) => void) | null = null;
    private isFlying: boolean = false;
    private lastPos: Vec3 = new Vec3();
    private isReturning: boolean = false; // 是否正在飞回
    private returnStartTime: Vec3 = new Vec3(); // 开始返回的位置
    private returnElapsedTime: number = 0; // 返回阶段的已用时间
    private returnTravelTime: number = 0; // 返回阶段的总飞行时间
    private ownerNode: Node = null!; // 女猎手节点（用于返回）
    private bounceCount: number = 0; // 当前弹射次数
    private enemiesHit: Set<Node> = new Set(); // 已命中的敌人集合
    private currentDamage: number = 0; // 当前伤害值
    private isBouncing: boolean = false; // 是否正在弹射
    private hasHitTarget: boolean = false; // 是否已命中目标
    private gameManager: GameManager | null = null; // 游戏管理器引用

    /**
     * 初始化回旋镖
     * @param startPos 起始位置
     * @param targetNode 目标节点
     * @param damage 伤害值
     * @param onHit 命中回调函数
     * @param ownerNode 发射者节点（女猎手）
     */
    init(startPos: Vec3, targetNode: Node, damage: number, onHit?: (damage: number) =>
void, ownerNode?: Node) {
        this.startPos = startPos.clone();
        this.targetNode = targetNode;
        this.damage = damage;
        this.currentDamage = damage; // 初始化当前伤害值
        this.onHitCallback = onHit || null;
        this.ownerNode = ownerNode || null!;

        // 设置初始状态
        this.isReturning = false;
        this.isBouncing = false;
        this.hasHitTarget = false; // 初始化命中标志
        this.bounceCount = 0;
        this.enemiesHit.clear(); // 清空已命中敌人集合
        this.returnStartTime = new Vec3();
        this.returnElapsedTime = 0;
        this.returnTravelTime = 0;

        // 设置初始位置
        this.node.setWorldPosition(this.startPos);

        // 计算目标位置
        if (this.targetNode && this.targetNode.isValid) {
            this.targetPos = this.targetNode.worldPosition.clone();
        } else {
            // 如果目标无效，使用起始位置前方
            this.targetPos = this.startPos.clone();
            this.targetPos.x += 200;
        }

        // 计算飞行时间（基于距离和速度）
```

```
        const distance = Vec3.distance(this.startPos, this.targetPos);
        if (distance < 0.1) {
            return;
        }

        this.travelTime = distance / this.speed;
        if (this.travelTime <= 0) {
            return;
        }

        this.elapsedTime = 0;
        this.isFlying = true;
        this.lastPos = this.startPos.clone();

        // 设置初始旋转角度（指向目标）
        const initialDirection = new Vec3();
        Vec3.subtract(initialDirection, this.targetPos, this.startPos);
        if (initialDirection.length() > 0.1) {
            const angle = Math.atan2(initialDirection.y, initialDirection.x) * 180 /
Math.PI;
            this.node.setRotationFromEuler(0, 0, angle);
        }
    }

    /**
     * 计算抛物线位置
     * @param ratio 进度比例 (0-1)
     */
    calculateParabolicPosition(ratio: number): Vec3 {
        const pos = new Vec3();

        // 线性插值计算基础位置
        Vec3.lerp(pos, this.startPos, this.targetPos, ratio);

        // 添加抛物线高度（使用二次函数模拟重力效果）
        const arcRatio = 4 * ratio * (1 - ratio); // 0 到 1 再到 0 的曲线
        pos.y += this.arcHeight * arcRatio;

        return pos;
    }

    /**
     * 命中目标
     */
    hitTarget() {

        if (this.hasHitTarget) {
            return; // 已经命中过了
        }

        if (!this.targetNode || !this.targetNode.isValid) {
            // 目标无效，开始返回
            this.startReturn();
            return;
        }

        this.hasHitTarget = true;

        // 直接对当前目标造成伤害，支持 Enemy、OrcWarrior、OrcWarlord 和 TrollSpearman
        const   enemyScript   =   this.targetNode.getComponent('Enemy')   as   any   ||
this.targetNode.getComponent('OrcWarrior')            as            any            ||
this.targetNode.getComponent('OrcWarlord')            as            any            ||
this.targetNode.getComponent('TrollSpearman') as any;
```

```
        if (enemyScript && typeof enemyScript.isAlive === 'function' && enemyScript.isAlive() && typeof enemyScript.takeDamage === 'function') {
            enemyScript.takeDamage(this.currentDamage);
        } else {
        }

        // 调用回调函数，用于播放音效等辅助效果
        if (this.onHitCallback) {
            this.onHitCallback(this.currentDamage);
        }

        // 将当前目标添加到已命中集合
        this.enemiesHit.add(this.targetNode);

        // 检查是否可以弹射
        if (this.bounceCount < this.maxBounces) {
            // 寻找附近的敌人
            const nearbyEnemies = this.findNearbyEnemies(this.targetNode, this.bounceRange);

            if (nearbyEnemies.length > 0) {
                // 选择最近的敌人作为下一个目标
                const nextTarget = this.findNearestEnemy(nearbyEnemies, this.targetNode.worldPosition);
                if (nextTarget) {
                    // 开始弹射
                    this.startBounce(nextTarget);
                    return;
                } else {
                }
            } else {
            }
        } else {
        }

        // 没有找到下一个目标或达到最大弹射次数，开始返回
        this.startReturn();
    }

    /**
     * 寻找附近的敌人
     * @param centerNode 中心点节点
     * @param range 搜索范围
     * @returns 附近的敌人数组
     */
    findNearbyEnemies(centerNode: Node, range: number): Node[] {

        const nearbyEnemies: Node[] = [];
        const centerPos = centerNode.worldPosition;

        // 查找 Enemies 容器，使用直接路径
        const enemiesNode = find('Canvas/Enemies');

        if (!enemiesNode) {
            return nearbyEnemies;
        }


        // 遍历所有敌人
        const enemies = enemiesNode.children || [];

        for (const enemy of enemies) {
            if (enemy && enemy.isValid && enemy.active && enemy !== centerNode) {
```

```
                // 检查敌人是否已经被命中
                if (this.enemiesHit.has(enemy)) {
                    continue;
                }

                // 检查敌人是否存活，支持 OrcWarlord、OrcWarrior、Enemy 和 TrollSpearman
                const enemyScript = enemy.getComponent('OrcWarlord') as any || enemy.getComponent('OrcWarrior') as any || enemy.getComponent('Enemy') as any || enemy.getComponent('TrollSpearman') as any;
                const isAlive = enemyScript && typeof enemyScript.isAlive === 'function' && enemyScript.isAlive();

                if (isAlive) {
                    // 检查距离
                    const enemyPos = enemy.worldPosition;
                    const distance = Vec3.distance(centerPos, enemyPos);

                    if (distance <= range) {
                        nearbyEnemies.push(enemy);
                    }
                }
            }
        }

        return nearbyEnemies;
    }

    /**
     * 寻找最近的敌人
     * @param enemies 敌人数组
     * @param currentPos 当前位置
     * @returns 最近的敌人
     */
    findNearestEnemy(enemies: Node[], currentPos: Vec3): Node | null {
        if (enemies.length === 0) {
            return null;
        }

        let nearestEnemy: Node | null = null;
        let minDistance = Infinity;

        for (const enemy of enemies) {
            if (enemy && enemy.isValid) {
                const distance = Vec3.distance(currentPos, enemy.worldPosition);

                if (distance < minDistance) {
                    minDistance = distance;
                    nearestEnemy = enemy;
                }
            }
        }

        return nearestEnemy;
    }

    /**
     * 递归查找节点
     * @param node 起始节点
     * @param name 节点名称
     * @returns 找到的节点
     */
    findNodeRecursive(node: Node, name: string): Node | null {
        if (node.name === name) {
```

```
            return node;
        }
        for (const child of node.children) {
            const found = this.findNodeRecursive(child, name);
            if (found) {
                return found;
            }
        }
        return null;
    }

    /**
     * 开始弹射
     * @param nextTarget 下一个目标
     */
    startBounce(nextTarget: Node) {
        this.isBouncing = true;
        this.bounceCount++;
        this.targetNode = nextTarget;
        this.targetPos = nextTarget.worldPosition.clone();
        this.currentDamage *= this.bounceDamageMultiplier; // 更新伤害值
        this.elapsedTime = 0; // 重置计时器
        this.hasHitTarget = false; // 重置命中标志，允许命中新目标

        // 计算新的飞行时间
        const distance = Vec3.distance(this.node.worldPosition, this.targetPos);
        this.travelTime = distance / this.speed;
        if (this.travelTime <= 0) {
            this.travelTime = 0.5; // 最小飞行时间
        }
    }

    /**
     * 开始返回女猎手手中
     */
    startReturn() {
        if (!this.ownerNode || !this.ownerNode.isValid) {
            // 如果没有女猎手节点，直接销毁
            this.scheduleOnce(() => {
                if (this.node && this.node.isValid) {
                    this.node.destroy();
                }
            }, 0.1);
            return;
        }

        // 重置命中标志，允许再次命中
        this.hasHitTarget = false;

        this.isReturning = true;
        this.returnStartTime = this.node.worldPosition.clone();
        this.returnElapsedTime = 0;

        // 计算返回的飞行时间
        const returnDistance = Vec3.distance(this.returnStartTime,
this.ownerNode.worldPosition);
        this.returnTravelTime = returnDistance / this.speed;
        if (this.returnTravelTime <= 0) {
            this.returnTravelTime = 0.5; // 最小返回时间
        }
    }

    /**
```

```
     * 返回女猎手手中完成
     */
    returnComplete() {
        this.isFlying = false;
        this.isReturning = false;

        // 销毁弓箭节点
        this.scheduleOnce(() => {
            if (this.node && this.node.isValid) {
                this.node.destroy();
            }
        }, 0.1);
    }

    /**
     * 计算返回路径位置
     * @param ratio 进度比例 (0-1)
     */
    calculateReturnPosition(ratio: number): Vec3 {
        const pos = new Vec3();

        // 计算返回的目标位置（女猎手位置）
        const returnTargetPos = this.ownerNode ? this.ownerNode.worldPosition.clone() :
this.startPos.clone();

        // 线性插值计算基础位置
        Vec3.lerp(pos, this.returnStartTime, returnTargetPos, ratio);

        // 添加返回时的抛物线高度
        const arcRatio = 4 * ratio * (1 - ratio); // 0 到 1 再到 0 的曲线
        pos.y += this.arcHeight * arcRatio * 0.5; // 返回时弧度减半

        return pos;
    }

    update(deltaTime: number) {
        if (!this.isFlying) {
            return;
        }

        // 检查游戏状态 - 如果 GameManager 不存在，尝试重新查找
        if (!this.gameManager) {
            this.gameManager = find('GameManager')?.getComponent(GameManager);
        }

        // 检查游戏状态，如果不是 Playing 状态，停止飞行
        if (this.gameManager) {
            const gameState = this.gameManager.getGameState();
            if (gameState !== GameState.Playing) {
                // 游戏已暂停或结束，停止飞行
                return;
            }
        }

        if (this.isReturning) {
            // 返回阶段
            this.returnElapsedTime += deltaTime;
            const returnRatio = Math.min(this.returnElapsedTime / this.returnTravelTime,
1);

            // 计算当前返回位置
            const currentPos = this.calculateReturnPosition(returnRatio);
```

```
            // 检查路径上是否有兽人督军尸体
            if (this.checkForOrcWarlordCorpse(this.lastPos, currentPos)) {
                return;
            }

            this.node.setWorldPosition(currentPos);

            // 更新旋转角度，使箭头始终指向飞行方向
            const direction = new Vec3();
            Vec3.subtract(direction, currentPos, this.lastPos);
            if (direction.length() > 0.1) {
                const angle = Math.atan2(direction.y, direction.x) * 180 / Math.PI;
                this.node.setRotationFromEuler(0, 0, angle);
            }
            this.lastPos = currentPos.clone();

            // 检查是否返回女猎手手中
            if (this.ownerNode && this.ownerNode.isValid) {
                const        distanceToOwner        =        Vec3.distance(currentPos,
this.ownerNode.worldPosition);
                if (distanceToOwner < 20) {
                    // 返回完成
                    this.returnComplete();
                    return;
                }
            }

            // 如果返回时间过长，强制销毁
            if (this.returnElapsedTime > this.returnTravelTime * 2) {
                this.returnComplete();
                return;
            }

            return;
        }

        // 正常飞行或弹射阶段
        // 更新计时器
        this.elapsedTime += deltaTime;

        // 更新目标位置（目标可能在移动）
        if (this.targetNode && this.targetNode.isValid && this.targetNode.active) {
            this.targetPos = this.targetNode.worldPosition.clone();
        } else {
            // 目标无效，寻找下一个目标或开始返回
            if (this.bounceCount < this.maxBounces) {
                // 寻找附近的敌人
                const      nearbyEnemies      =      this.findNearbyEnemies(this.node,
this.bounceRange);
                if (nearbyEnemies.length > 0) {
                    // 选择最近的敌人作为下一个目标
                    const     nextTarget     =     this.findNearestEnemy(nearbyEnemies,
this.node.worldPosition);
                    if (nextTarget) {
                        // 开始弹射
                        this.startBounce(nextTarget);
                    } else {
                        // 没有找到下一个目标，开始返回
                        this.startReturn();
                    }
                } else {
                    // 没有附近敌人，开始返回
                    this.startReturn();
```

```
            }
            return;
        } else {
            // 达到最大弹射次数，开始返回
            this.startReturn();
            return;
        }
    }

    // 计算当前进度
    const currentRatio = Math.min(this.elapsedTime / this.travelTime, 1);

    // 计算当前在抛物线上的位置
    const currentPos = this.calculateParabolicPosition(currentRatio);

    // 检查路径上是否有兽人督军尸体
    if (this.checkForOrcWarlordCorpse(this.lastPos, currentPos)) {
        return;
    }

    this.node.setWorldPosition(currentPos);

    // 更新旋转角度，使箭头始终指向飞行方向
    const direction = new Vec3();
    Vec3.subtract(direction, currentPos, this.lastPos);
    if (direction.length() > 0.1) {
        const angle = Math.atan2(direction.y, direction.x) * 180 / Math.PI;
        this.node.setRotationFromEuler(0, 0, angle);
    }
    this.lastPos = currentPos.clone();

    // 检查是否命中目标
    if (this.targetNode && this.targetNode.isValid && this.targetNode.active) {
        const currentTargetPos = this.targetNode.worldPosition;
        const distance = Vec3.distance(currentPos, currentTargetPos);

        // 如果距离足够近，认为命中
        if (distance < 30) {
            this.hitTarget();
            return;
        }
    }

    // 如果飞行完成但未命中目标，尝试弹射或返回
    if (currentRatio >= 1) {
        this.hitTarget(); // 调用 hitTarget 来处理弹射或返回逻辑
        return;
    }

    // 如果飞行时间过长（超过预期时间 2 倍），开始返回
    if (this.elapsedTime > this.travelTime * 2) {
        this.hitTarget();
        return;
    }
}

/**
 * 将回旋镖插在尸体身上
 * @param corpseNode 尸体节点
 * @param startPos 起始位置
 * @param endPos 结束位置
 */
attachToCorpse(corpseNode: Node, startPos: Vec3, endPos: Vec3): void {
```

```
        if (!this.node || !this.node.isValid || !corpseNode || !corpseNode.isValid) {
            return;
        }

        // 计算命中点
        const corpsePos = corpseNode.worldPosition;
        const hitPos = this.calculateHitPoint(startPos, endPos, corpsePos);

        // 停止飞行状态
        this.isFlying = false;
        this.isReturning = false;

        // 将回旋镖设为尸体的子节点
        this.node.removeFromParent();
        corpseNode.addChild(this.node);

        // 设置回旋镖位置（转换为局部位置）
        const localPos = corpseNode.worldPosition.clone();
        Vec3.subtract(localPos, hitPos, localPos);
        this.node.setPosition(localPos);

        // 调整回旋镖旋转，使其指向飞行方向
        const direction = new Vec3();
        Vec3.subtract(direction, endPos, startPos);
        if (direction.length() > 0.1) {
            const angle = Math.atan2(direction.y, direction.x) * 180 / Math.PI;
            this.node.setRotationFromEuler(0, 0, angle);
        }

        // 命中尸体不触发伤害回调
        // 清除回调，防止后续调用
        this.onHitCallback = null;
    }

    /**
     * 计算命中点
     * @param startPos 起始位置
     * @param endPos 结束位置
     * @param corpsePos 尸体位置
     * @returns 命中点位置
     */
    calculateHitPoint(startPos: Vec3, endPos: Vec3, corpsePos: Vec3): Vec3 {
        // 计算线段方向
        const lineDir = Vec3.subtract(new Vec3(), endPos, startPos);
        const lineLengthSqr = Vec3.lengthSqr(lineDir);

        if (lineLengthSqr === 0) {
            return corpsePos.clone();
        }

        // 计算投影点
        const t = Math.max(0, Math.min(1, Vec3.dot(Vec3.subtract(new Vec3(), corpsePos,
startPos), lineDir) / lineLengthSqr));
        const projection = Vec3.add(new Vec3(), startPos, Vec3.multiplyScalar(new Vec3(),
lineDir, t));

        return projection;
    }

    /**
     * 检查路径上是否有兽人督军尸体
     * @param startPos 起始位置
     * @param endPos 结束位置
```

```
     * @returns 是否命中尸体
     */
    checkForOrcWarlordCorpse(startPos: Vec3, endPos: Vec3): boolean {
        // 查找 Enemies 容器，使用直接路径
        const enemiesNode = find('Canvas/Enemies');

        if (!enemiesNode) {
            return false;
        }

        const enemies = enemiesNode.children || [];

        for (const enemy of enemies) {
            if (!enemy || !enemy.isValid) {
                continue;
            }

            // 检查是否是兽人督军
            const orcScript = enemy.getComponent('OrcWarlord') as any;
            if (!orcScript) {
                continue;
            }

            // 检查是否已经死亡
            const isAlive = orcScript.isAlive && orcScript.isAlive();
            if (!isAlive && enemy.isValid) {
                // 检查整个飞行路径是否与尸体相交
                const corpsePos = enemy.worldPosition;
                const distance = this.getDistanceFromLine(startPos, endPos, corpsePos);

                // 如果距离小于 30 像素，认为命中尸体
                if (distance < 30) {
                    // 命中尸体，插在尸体身上
                    this.attachToCorpse(enemy, startPos, endPos);
                    return true;
                }
            }
        }

        return false;
    }

    /**
     * 计算点到线段的最短距离
     * @param lineStart 线段起点
     * @param lineEnd 线段终点
     * @param point 点
     * @returns 最短距离
     */
    getDistanceFromLine(lineStart: Vec3, lineEnd: Vec3, point: Vec3): number {
        const lineDir = Vec3.subtract(new Vec3(), lineEnd, lineStart);
        const lineLengthSqr = Vec3.lengthSqr(lineDir);

        if (lineLengthSqr === 0) {
            // 线段长度为 0，直接返回点到起点的距离
            return Vec3.distance(point, lineStart);
        }

        const t = Math.max(0, Math.min(1, Vec3.dot(Vec3.subtract(new Vec3(), point,
lineStart), lineDir) / lineLengthSqr));
        const projection = Vec3.add(new Vec3(), lineStart, Vec3.multiplyScalar(new
Vec3(), lineDir, t));
```

```
        return Vec3.distance(point, projection);
    }
}

//中间已省略多余部分源代码
//中间已省略多余部分源代码
//中间已省略多余部分源代码

    /**
     * 显示占领指示器
     */
    private showCaptureIndicator(isFriendly: boolean) {
        if (this.captureIndicator && this.captureIndicator.isValid) {
            return; // 已经显示
        }

        // 隐藏之前的指示器
        this.hideCaptureIndicator();

        // 创建指示器节点
        this.captureIndicator = new Node('CaptureIndicator');
        this.captureIndicator.setParent(this.node);
        this.captureIndicator.setPosition(0, 0, 0);

        // 添加 UITransform
        const uiTransform = this.captureIndicator.addComponent(UITransform);
        uiTransform.setContentSize(this.attackRange * 2, this.attackRange * 2);

        // 创建 Graphics 组件用于绘制圆弧
        this.captureIndicatorGraphics = this.captureIndicator.addComponent(Graphics);
    }

    /**
     * 更新占领指示器进度
     */
    private updateCaptureIndicator(progress: number, isFriendly: boolean) {
        if      (!this.captureIndicator      ||      !this.captureIndicator.isValid
|| !this.captureIndicatorGraphics) {
            return;
        }

        // 清除之前的绘制
        this.captureIndicatorGraphics.clear();

        // 设置绘制参数
        const radius = this.attackRange; // 使用攻击范围作为半径
        const lineWidth = 6; // 线条宽度
        const centerX = 0;
        const centerY = 0;

        // 确保进度在 0-1 之间
        const clampedProgress = Math.max(0, Math.min(1, progress));

        // 如果进度为 0，不绘制任何内容
        if (clampedProgress <= 0) {
            return;
        }

        // 计算圆弧的起始角度和结束角度（使用弧度制）
        // 从顶部（-90 度）开始，顺时针绘制
        const endAngle = -Math.PI / 2; // 从顶部开始（-90 度 = -π/2）
        // 结束角度 = 起始角度 + 进度 * 360 度（顺时针）
        const startAngle = endAngle + clampedProgress * Math.PI * 2;
```

```
        // 根据占领方设置颜色
        if (isFriendly) {
            // 我方：绿色
            this.captureIndicatorGraphics.strokeColor = new Color(100, 255, 100, 200);
        } else {
            // 敌方：红色
            this.captureIndicatorGraphics.strokeColor = new Color(255, 100, 100, 200);
        }
        this.captureIndicatorGraphics.lineWidth = lineWidth;

        // 绘制圆弧（从 startAngle 到 endAngle，顺时针）
        this.captureIndicatorGraphics.arc(centerX,   centerY,   radius,   startAngle,
endAngle, false);
        this.captureIndicatorGraphics.stroke();
    }

    /**
     * 隐藏占领指示器
     */
    private hideCaptureIndicator() {
        if (this.captureIndicator && this.captureIndicator.isValid) {
            this.captureIndicator.destroy();
            this.captureIndicator = null!;
            this.captureIndicatorGraphics = null!;
        }
    }

    /**
     * 查找攻击目标（根据占领状态攻击不同目标）
     */
    private findTarget() {
        if (this.isDestroyed) {
            return;
        }

        const myPos = this.node.worldPosition;
        let nearestTarget: Node | null = null;
        let minDistance = this.attackRange;

        // 根据占领状态选择目标
        if (this.captureState === CaptureState.Friendly) {
            // 我方占领，攻击敌人
            nearestTarget = this.findNearestEnemy(myPos, minDistance);
        } else if (this.captureState === CaptureState.Enemy) {
            // 敌方占领，攻击我方单位
            nearestTarget = this.findNearestFriendlyUnit(myPos, minDistance);
        } else {
            // 中立状态，攻击敌人（默认行为）
            nearestTarget = this.findNearestEnemy(myPos, minDistance);
        }

        // 设置当前目标
        if (nearestTarget) {
            this.currentTarget = nearestTarget;
        } else {
            this.currentTarget = null!;
        }
    }

    /**
     * 查找最近的敌人
     */
```

```
    private findNearestEnemy(center: Vec3, maxRange: number): Node | null {
        let nearestTarget: Node | null = null;
        let minDistance = maxRange;

        // 使用 UnitManager 优化
        if (this.unitManager) {
            const enemies = this.unitManager.getEnemiesInRange(center, maxRange, true);
            for (const enemy of enemies) {
                const distance = Vec3.distance(center, enemy.worldPosition);
                if (distance < minDistance) {
                    minDistance = distance;
                    nearestTarget = enemy;
                }
            }
            return nearestTarget;
        }

        // 降级方案：直接查找
        const      enemyContainers      =      ['Canvas/Enemies',      'Canvas/Orcs',
'Canvas/TrollSpearmans', 'Canvas/OrcWarriors', 'Canvas/OrcWarlords'];

        for (const containerName of enemyContainers) {
            const containerNode = find(containerName);
            if (containerNode) {
                for (const enemy of containerNode.children) {
                    if (!enemy || !enemy.isValid || !enemy.active) continue;
                    const enemyScript = enemy.getComponent('Enemy') as any ||
                                        enemy.getComponent('OrcWarlord') as any ||
                                        enemy.getComponent('OrcWarrior') as any ||
                                        enemy.getComponent('TrollSpearman') as any;
                    if (!enemyScript || !enemyScript.isAlive || !enemyScript.isAlive())
continue;
                    const distance = Vec3.distance(center, enemy.worldPosition);
                    if (distance <= maxRange && distance < minDistance) {
                        minDistance = distance;
                        nearestTarget = enemy;
                    }
                }
            }
        }

        return nearestTarget;
    }

    /**
     * 查找最近的我方单位
     */
    private findNearestFriendlyUnit(center: Vec3, maxRange: number): Node | null {
        let nearestTarget: Node | null = null;
        let minDistance = maxRange;

        // 查找 Towers 容器（包含 Arrower 和 Priest）
        const towersNode = find('Canvas/Towers');
        if (towersNode) {
            for (const unit of towersNode.children) {
                if (!unit || !unit.isValid || !unit.active) continue;
                const arrowerScript = unit.getComponent('Arrower') as any;
                const priestScript = unit.getComponent('Priest') as any;
                const unitScript = arrowerScript || priestScript;
                if  (!unitScript  ||  !unitScript.isAlive  ||  !unitScript.isAlive())
continue;
                const distance = Vec3.distance(center, unit.worldPosition);
                if (distance <= maxRange && distance < minDistance) {
```

```
                    minDistance = distance;
                    nearestTarget = unit;
                }
            }
        }

        // 查找 Hunters 容器
        const huntersNode = find('Canvas/Hunters');
        if (huntersNode) {
            for (const unit of huntersNode.children) {
                if (!unit || !unit.isValid || !unit.active) continue;
                const unitScript = unit.getComponent('Hunter') as any;
                if  (!unitScript  ||  !unitScript.isAlive  ||  !unitScript.isAlive())
continue;
                const distance = Vec3.distance(center, unit.worldPosition);
                if (distance <= maxRange && distance < minDistance) {
                    minDistance = distance;
                    nearestTarget = unit;
                }
            }
        }

        // 查找 ElfSwordsmans 容器
        const swordsmansNode = find('Canvas/ElfSwordsmans');
        if (swordsmansNode) {
            for (const unit of swordsmansNode.children) {
                if (!unit || !unit.isValid || !unit.active) continue;
                const unitScript = unit.getComponent('ElfSwordsman') as any;
                if  (!unitScript  ||  !unitScript.isAlive  ||  !unitScript.isAlive())
continue;
                const distance = Vec3.distance(center, unit.worldPosition);
                if (distance <= maxRange && distance < minDistance) {
                    minDistance = distance;
                    nearestTarget = unit;
                }
            }
        }

        return nearestTarget;
    }

    /**
     * 攻击目标
     */
    private attack() {
        if (!this.currentTarget || this.isDestroyed) {
            return;
        }

        // 再次检查目标是否有效
        if (!this.currentTarget.isValid || !this.currentTarget.active) {
            this.currentTarget = null!;
            return;
        }

        // 根据占领状态获取不同的脚本
        // 注释掉占领逻辑，哨塔只能归属我方，始终攻击敌人
        let targetScript: any = null;
        // if (this.captureState === CaptureState.Enemy) {
        //     // 敌方占领，攻击我方单位
        //     targetScript = this.currentTarget.getComponent('Arrower') as any ||
        //                    this.currentTarget.getComponent('Hunter') as any ||
        //                    this.currentTarget.getComponent('ElfSwordsman') as any ||
```

```
        //                  this.currentTarget.getComponent('Priest') as any;
        // } else {
            // 我方占领或中立，攻击敌人（哨塔只能归属我方）
            targetScript = this.currentTarget.getComponent('Enemy') as any ||
                           this.currentTarget.getComponent('OrcWarlord') as any ||
                           this.currentTarget.getComponent('OrcWarrior') as any ||
                           this.currentTarget.getComponent('TrollSpearman') as any;
        // }

        if (!targetScript || !targetScript.isAlive || !targetScript.isAlive()) {
            this.currentTarget = null!;
            return;
        }

        // 播放射击音效
        if (this.shootSound && AudioManager.Instance) {
            AudioManager.Instance.playSFX(this.shootSound);
        }

        // 创建弓箭
        if (this.arrowPrefab) {
            this.createArrow();
        } else {
            // 如果没有弓箭预制体，直接造成伤害
            if (targetScript.takeDamage) {
                targetScript.takeDamage(this.attackDamage);
                // 记录伤害统计
                this.recordDamageToStatistics(this.attackDamage);
            }
        }
    }

    /**
     * 创建弓箭
     */
    private createArrow() {
        if (!this.arrowPrefab || !this.currentTarget) {
            return;
        }

        // 检查目标是否有效
        if (!this.currentTarget.isValid || !this.currentTarget.active) {
            return;
        }

        // 创建弓箭节点
        const arrow = instantiate(this.arrowPrefab);

        // 设置父节点（添加到场景或Canvas）
        const canvas = find('Canvas');
        const scene = this.node.scene;
        const parentNode = canvas || scene || this.node.parent;
        if (parentNode) {
            arrow.setParent(parentNode);
        } else {
            arrow.setParent(this.node.parent);
        }

        // 设置初始位置（哨塔位置）
        const startPos = this.node.worldPosition.clone();
        arrow.setWorldPosition(startPos);

        // 确保节点激活
```

```
        arrow.active = true;

        // 获取或添加 Arrow 组件
        let arrowScript = arrow.getComponent(Arrow);
        if (!arrowScript) {
            arrowScript = arrow.addComponent(Arrow);
        }

        // 保存当前目标的引用，避免回调函数中引用失效的目标
        const targetNode = this.currentTarget;

        // 初始化弓箭，设置命中回调
        arrowScript.init(
            startPos,
            targetNode,
            this.attackDamage,
            (damage: number) => {
                // 播放箭矢击中音效
                if (this.hitSound) {
                    AudioManager.Instance?.playSFX(this.hitSound);
                }

                // 检查目标是否仍然有效
                if (targetNode && targetNode.isValid && targetNode.active) {
                    // 根据占领状态获取不同的脚本
                    let targetScript: any = null;
                    if (this.captureState === CaptureState.Enemy) {
                        targetScript = targetNode.getComponent('Arrower') as any ||
                                       targetNode.getComponent('Hunter') as any ||
                                       targetNode.getComponent('ElfSwordsman') as any ||
                                       targetNode.getComponent('Priest') as any;
                    } else {
                        targetScript = targetNode.getComponent('Enemy') as any ||
                                        targetNode.getComponent('OrcWarlord') as any ||
                                        targetNode.getComponent('OrcWarrior') as any ||
                                        targetNode.getComponent('TrollSpearman') as any;
                    }

                    if (targetScript && targetScript.takeDamage) {
                        targetScript.takeDamage(damage);
                        // 记录伤害统计（哨塔属于建筑，使用 Build 里的方法）
                        this.recordDamageToStatistics(damage);
                    }
                }
            }
        );
    }

    /**
     * 死亡（重写为：进入被破坏阶段，后续播放风化并最终回收）
     */
    protected die() {
        if (this.isDestroyed || this.isDestroying) {
            return;
        }

        this.isDestroyed = true;
        this.isDestroying = true;
        this.destructionTimer = 0;
        this.destructionFrameIndex = 0;

        // 释放人口
        if (!this.gameManager) {
```

```
            this.findGameManager();
        }
        if (this.gameManager) {
            this.gameManager.removePopulation(this.populationCost);
        }

        // 停止攻击
        this.currentTarget = null!;

        // 隐藏占领指示器
        this.hideCaptureIndicator();

        // 隐藏血条
        if (this.healthBarNode) {
            this.healthBarNode.active = false;
        }

        // 更新贴图（进入被破坏帧 1）
        this.updateSprite();

        // 播放爆炸特效（如果有）
        if (this.explosionEffect) {
            const explosion = instantiate(this.explosionEffect);
            const canvas = find('Canvas');
            if (canvas) {
                explosion.setParent(canvas);
            } else if (this.node.scene) {
                explosion.setParent(this.node.scene);
            }
            explosion.setWorldPosition(this.node.worldPosition);
            explosion.active = true;
        }
    }

    /**
     * 最终销毁（风化完成后调用）
     */
    private finalDestroy() {
        // 释放网格占用（确保能找到网格面板）
        if (!this.gridPanel) {
            this.findGridPanel();
        }
        if (this.gridPanel && this.gridX >= 0 && this.gridY >= 0) {
            // 哨塔占据两个网格高度，需要释放两个网格
            this.gridPanel.releaseGrid(this.gridX, this.gridY);
            const gridPanel = this.gridPanel as any;
            if (this.gridY + 1 < gridPanel.gridHeight) {
                this.gridPanel.releaseGrid(this.gridX, this.gridY + 1);
            }
        }

        // 给玩家奖励金币（哨塔特有）
        if (this.gameManager) {
            this.gameManager.addGold(this.goldReward);
        }

        // 性能优化：使用对象池回收建筑物，而不是直接销毁
        const returnToPool = () => {
            const buildingPool = BuildingPool.getInstance();
            if (buildingPool && this.node && this.node.isValid) {
                this.resetBuildingState();
                buildingPool.release(this.node, this.prefabName);
            } else {
```

```
            if (this.node && this.node.isValid) {
                this.node.destroy();
            }
        }
    };

    this.scheduleOnce(() => {
        returnToPool();
    }, 0.1);
}

/**
 * 创建建造进度条
 */
private createConstructionProgressBar() {
    if (this.constructionProgressBar && this.constructionProgressBar.isValid) {
        return;
    }

    const barNode = new Node('ConstructionProgressBar');
    barNode.setParent(this.node);
    barNode.setPosition(0, 60, 0);

    const uiTransform = barNode.addComponent(UITransform);
    uiTransform.setContentSize(60, 6);

    const graphics = barNode.addComponent(Graphics);
    graphics.lineWidth = 2;

    this.constructionProgressBar = barNode;
    this.constructionProgressGraphics = graphics;

    barNode.active = true;
}

/**
 * 更新建造进度条显示
 */
private updateConstructionProgressBar() {
    if (!this.constructionProgressBar || !this.constructionProgressGraphics) {
        return;
    }

    if (this.constructionStage >= ConstructionStage.COMPLETE) {
        this.constructionProgressBar.active = false;
        return;
    }

    this.constructionProgressBar.active = true;

    const g = this.constructionProgressGraphics;
    g.clear();

    const width = 60;
    const height = 6;

    g.strokeColor = new Color(0, 0, 0, 255);
    g.lineWidth = 1;
    g.rect(-width / 2, -height / 2, width, height);
    g.stroke();

    const progressWidth = width * Math.max(0, Math.min(1,
this.constructionProgress));
```

```
        g.fillColor = new Color(0, 200, 0, 255);
        g.rect(-width / 2 + 1, -height / 2 + 1, progressWidth - 2, height - 2);
        g.fill();
    }

    /**
     * 根据当前阶段更新贴图和高度
     */
    private updateSprite() {
        if (!this.sprite) {
            const s = this.node.getComponent(Sprite);
            if (s) {
                this.sprite = s;
            } else {
                return;
            }
        }

        // 优先处理风化阶段
        if (this.weatheringStage === WeatheringStage.STAGE_1 && this.weatheringSprite1)
{
            this.sprite.spriteFrame = this.weatheringSprite1;
            this.setHeightWithFixedBottomGeneric(0.5);
            return;
        }
        if (this.weatheringStage === WeatheringStage.STAGE_2 && this.weatheringSprite2)
{
            this.sprite.spriteFrame = this.weatheringSprite2;
            this.setHeightWithFixedBottomGeneric(0.5);
            return;
        }
        if (this.weatheringStage === WeatheringStage.STAGE_3 && this.weatheringSprite3)
{
            this.sprite.spriteFrame = this.weatheringSprite3;
            this.setHeightWithFixedBottomGeneric(0.5);
            return;
        }

        // 处理被破坏阶段
        if (this.isDestroying) {
            if (this.destructionFrameIndex === 0 && this.destructionSprite1) {
                this.sprite.spriteFrame = this.destructionSprite1;
                return;
            }
            if (this.destructionFrameIndex === 1 && this.destructionSprite2) {
                this.sprite.spriteFrame = this.destructionSprite2;
                return;
            }
            if (this.destructionFrameIndex === 2 && this.destructionSprite3) {
                this.sprite.spriteFrame = this.destructionSprite3;
                return;
            }
        }

        // 处理建造阶段
        if     (this.constructionStage     ===     ConstructionStage.FOUNDATION     &&
this.foundationSprite) {
            this.sprite.spriteFrame = this.foundationSprite;
            this.setHeightWithFixedBottomGeneric(0.5);
            return;
        }
        if     (this.constructionStage     ===     ConstructionStage.HALF_BUILT     &&
this.halfBuiltSprite) {
```

```
            this.sprite.spriteFrame = this.halfBuiltSprite;
            this.setHeightWithFixedBottomGeneric(0.66);
            return;
        }
        if     (this.constructionStage     ===     ConstructionStage.COMPLETE     &&
this.completeSprite) {
            this.sprite.spriteFrame = this.completeSprite;
            this.setHeightWithFixedBottomGeneric(1.0);
            return;
        }

        // 如果没有配置贴图，默认按高度阶段缩放
        if (this.constructionStage === ConstructionStage.COMPLETE) {
            this.setHeightWithFixedBottomGeneric(1.0);
        } else if (this.constructionStage === ConstructionStage.HALF_BUILT) {
            this.setHeightWithFixedBottomGeneric(0.66);
        } else if (this.constructionStage === ConstructionStage.FOUNDATION) {
            this.setHeightWithFixedBottomGeneric(0.5);
        }
    }

    // 具体实现复用父类 Build 的 setHeightWithFixedBottomGeneric

    /**
     * 检查是否存活
     */
    public isAlive(): boolean {
        return !this.isDestroyed && !this.isDestroying && this.weatheringStage ===
WeatheringStage.NONE && this.currentHealth > 0;
    }
}


// ==============================
// File: assets/scripts/role/Wisp.ts
// ==============================
import { _decorator, Component, Node, Vec2, Vec3, Prefab, instantiate, find, Graphics,
UITransform, Label, Color, EventTouch, Sprite, SpriteFrame, resources } from 'cc';
import { GameManager, GameState } from '../GameManager';
import { HealthBar } from '../HealthBar';
import { DamageNumber } from '../DamageNumber';
import { UnitSelectionManager } from '../UnitSelectionManager';
import { UnitInfo } from '../UnitInfoPanel';
import { GamePopup } from '../GamePopup';
import { Role } from './Role';
import { ForestGridPanel } from '../ForestGridPanel';
import { UnitType } from '../UnitType';
const { ccclass, property } = _decorator;

@ccclass('Wisp')
export class Wisp extends Role {
    @property(Prefab)
    healEffectPrefab: Prefab = null!; // 治疗特效预制体

    @property
    healAmount: number = 2; // 每次治疗恢复的血量

    @property
    healInterval: number = 0.5; // 治疗间隔（秒），改为每 0.5 秒维修一次

    @property
    attachOffset: Vec3 = new Vec3(0, 30, 0); // 依附在建筑物上的偏移位置
```

```
    @property
    repairRange: number = 20; // 维修范围（必须在 20 以内才能维修）

    @property
    searchRange: number = 200; // 搜索范围（自动寻找 200 范围内的建筑物）

    @property({ type: [SpriteFrame] })
    repairAnimationFrames: SpriteFrame[] = []; // 维修动画帧数组

    @property
    repairAnimationDuration: number = 1.0; // 维修动画时长（秒），调慢一半

    // 依附相关
    private attachedBuilding: Node = null!; // 依附的建筑物
    private isAttached: boolean = false; // 是否已依附
    private healTimer: number = 0; // 治疗计时器

    // 维修相关
    private repairTarget: Node = null!; // 当前维修目标（防御塔或石墙）
    private isRepairing: boolean = false; // 是否正在维修
    private manualControlActive: boolean = false; // 手动控制激活标志
    private isPlayingRepairAnimation: boolean = false; // 是否正在播放维修动画

    // 树林隐蔽效果相关
    private isInForestArea: boolean = false;
    private originalSpriteColor: Color | null = null;

    start() {
        // 设置单位类型为 CHARACTER
        this.unitType = UnitType.CHARACTER;

        // 调用父类初始化（血量、血条、选择、对话等）
        super.start();

        // 小精灵是辅助单位，没有攻击能力
        this.attackDamage = 0;
        this.attackRange = 0;

        // 初始化依附状态
        this.isAttached = false;
        this.attachedBuilding = null!;
        this.healTimer = 0;

        // 初始化维修状态
        this.isRepairing = false;
        this.repairTarget = null!;
        this.manualControlActive = false;
        this.isPlayingRepairAnimation = false;

        // 设置默认名称（如果未在编辑器中配置）
        if (!this.unitName || this.unitName === '') {
            this.unitName = '小精灵';
        }

        // 小精灵的 Sprite 使用自定义尺寸并启用裁剪
        if (this.sprite) {
            this.sprite.sizeMode = Sprite.SizeMode.CUSTOM;
            this.sprite.trim = true;
            this.originalSpriteColor = this.sprite.color.clone();
        }

        // 如果伤害数字预制体未设置，尝试加载
        if (!this.damageNumberPrefab) {
```

```
            this.loadDamageNumberPrefab();
        }
    }

    /**
     * 加载伤害数字预制体
     */
    private loadDamageNumberPrefab() {
        resources.load('prefabs/DamageNumber', Prefab, (err, prefab) => {
            if (!err && prefab) {
                this.damageNumberPrefab = prefab;
            } else {
                console.warn('[Wisp] Failed to load DamageNumber prefab:', err);
            }
        });
    }

    onDestroy() {
        // 减少人口
        if (this.gameManager) {
            this.gameManager.removePopulation(1);
        }
    }

    update(deltaTime: number) {
        if (this.isDestroyed) {
            return;
        }

        // 在调用父类 update 之前，检查并保存手动控制状态
        const      wasManuallyControlled      =      this.isManuallyControlled      ||
this.manualMoveTarget != null;

        if (wasManuallyControlled) {
            if (!this.manualControlActive) {
                console.log('[Wisp] ===== MANUAL CONTROL ACTIVATED =====');
                console.log('[Wisp]                            isManuallyControlled:',
this.isManuallyControlled);
                console.log('[Wisp] manualMoveTarget:', this.manualMoveTarget);
                this.manualControlActive = true;
                this.repairTarget = null!;
                this.isRepairing = false;
            }
        }

        // 调用父类处理移动、选中、碰撞等通用逻辑
        super.update(deltaTime);

        // 在父类 update 之后，再次检查手动控制状态
        // 如果之前是手动控制，但现在标志被清空了，保持 manualControlActive 为 true
        const     isStillManuallyControlled     =     this.isManuallyControlled     ||
this.manualMoveTarget != null;

        if (!isStillManuallyControlled && this.manualControlActive) {
            // 父类可能已经完成了移动，现在可以停用手动控制
            console.log('[Wisp] ===== MANUAL CONTROL DEACTIVATED =====');
            this.manualControlActive = false;
        }

        // 移除依附功能，只保留维修功能
        if (!this.manualControlActive) {
            // 只有在非手动控制状态下才执行维修逻辑
            this.updateRepairLogic(deltaTime);
```

```
        }

        // 更新树林隐蔽效果
        this.updateForestStealthState();
    }

    /**
     * 更新维修逻辑
     */
    private updateRepairLogic(deltaTime: number) {
        // 如果正在手动控制移动，不执行维修逻辑（手动控制优先级最高）
        if (this.isManuallyControlled || this.manualMoveTarget) {
            this.repairTarget = null!;
            this.isRepairing = false;
            return;
        }

        // 更新维修计时器
        this.healTimer += deltaTime;

        // 查找需要维修的目标
        if         (!this.repairTarget         ||         !this.repairTarget.isValid
|| !this.repairTarget.active) {
            this.findRepairTarget();
        }

        // 如果有维修目标
        if (this.repairTarget) {
            const       distance       =       Vec3.distance(this.node.worldPosition,
this.repairTarget.worldPosition);

            // 如果在维修范围内
            if (distance <= this.repairRange) {
                // 不要设置 isMoving = false，让父类的移动逻辑继续工作
                this.isRepairing = true;

                // 执行维修
                if (this.healTimer >= this.healInterval) {
                    this.healTimer = 0;
                    this.repairBuilding();
                }
            } else if (distance <= this.searchRange) {
                // 如果在搜索范围内但不在维修范围内，移动到目标附近
                this.isRepairing = true;
                this.moveToRepairTarget(this.repairTarget, deltaTime);
            } else {
                // 目标超出搜索范围，清除目标
                this.repairTarget = null!;
                this.isRepairing = false;
            }
        } else {
            this.isRepairing = false;
        }
    }

    /**
     * 查找需要维修的目标（防御塔或石墙）
     */
    private findRepairTarget() {
        const containers = [
            'Canvas/Towers',
            'Canvas/WarAncientTrees',
            'Canvas/HunterHalls',
```

```
            'Canvas/StoneWalls',
            'Canvas/WatchTowers',
            'Canvas/IceTowers',
            'Canvas/ThunderTowers',
            'Canvas/SwordsmanHalls',
            'Canvas/Churches'
        ];

        let nearestTarget: Node = null!;
        let minDistance = Infinity;

        for (const containerPath of containers) {
            const container = this.findNodeByPath(containerPath);
            if (!container) continue;

            for (const building of container.children) {
                if (!building || !building.active || !building.isValid) continue;

                // 获取建筑物脚本
                const buildingScript = this.getBuildingScript(building);
                if (!buildingScript) continue;

                // 检查建筑物是否存活
                if (buildingScript.isAlive && !buildingScript.isAlive()) continue;

                // 检查是否需要维修（生命值未满）
                const     currentHealth     =     buildingScript.currentHealth     ||
buildingScript.getHealth?.();
                const       maxHealth       =       buildingScript.maxHealth       ||
buildingScript.getMaxHealth?.();

                if  (currentHealth  !==  undefined  &&  maxHealth  !==  undefined  &&
currentHealth < maxHealth) {
                    const     distance     =     Vec3.distance(this.node.worldPosition,
building.worldPosition);
                    if (distance <= this.searchRange && distance < minDistance) {
                        minDistance = distance;
                        nearestTarget = building;
                    }
                }
            }
        }

        this.repairTarget = nearestTarget;
    }

    /**
     * 维修建筑物
     */
    private repairBuilding() {
        if (!this.repairTarget || this.isDestroyed) {
            return;
        }

        if (!this.repairTarget.isValid || !this.repairTarget.active) {
            this.repairTarget = null!;
            this.isRepairing = false;
            return;
        }

        // 获取建筑物脚本
        const buildingScript = this.getBuildingScript(this.repairTarget);
        if (!buildingScript) {
```

```
            this.repairTarget = null!;
            this.isRepairing = false;
            return;
        }

        // 检查建筑物是否存活
        if (buildingScript.isAlive && !buildingScript.isAlive()) {
            this.repairTarget = null!;
            this.isRepairing = false;
            return;
        }

        // 获取当前生命值和最大生命值
        const currentHealth = buildingScript.currentHealth || buildingScript.getHealth?.();
        const maxHealth = buildingScript.maxHealth || buildingScript.getMaxHealth?.();

        // 检查是否需要维修
        if (currentHealth >= maxHealth) {
            this.repairTarget = null!;
            this.isRepairing = false;
            return;
        }

        // 播放维修动画
        this.playRepairAnimation();

        // 恢复生命值
        const healAmount = Math.min(this.healAmount, maxHealth - currentHealth);

        if (buildingScript.heal) {
            buildingScript.heal(healAmount);
        } else if (buildingScript.currentHealth !== undefined) {
            buildingScript.currentHealth = Math.min(maxHealth, currentHealth + healAmount);
            // 更新血条
            if (buildingScript.updateHealthBar) {
                buildingScript.updateHealthBar();
            } else if (buildingScript.healthBar) {
                buildingScript.healthBar.setHealth(buildingScript.currentHealth);
            }
        }

        // 显示治疗特效和数字
        this.showRepairEffect(this.repairTarget);
        this.showRepairNumber(this.repairTarget, healAmount);
    }

    /**
     * 播放维修动画
     */
    private playRepairAnimation() {
        if (this.isPlayingRepairAnimation) {
            return;
        }

        if (!this.sprite) {
            this.sprite = this.node.getComponent(Sprite);
            if (!this.sprite) {
                return;
            }
        }
```

```
        if (!this.repairAnimationFrames || this.repairAnimationFrames.length === 0) {
            return;
        }

        const validFrames = this.repairAnimationFrames.filter(frame => frame != null);
        if (validFrames.length === 0) {
            return;
        }

        this.isPlayingRepairAnimation = true;

        const frames = validFrames;
        const frameCount = frames.length;
        const frameDuration = this.repairAnimationDuration / frameCount;
        let animationTimer = 0;
        let lastFrameIndex = -1;

        if (frames[0]) {
            this.sprite.spriteFrame = frames[0];
            lastFrameIndex = 0;
        }

        const animationUpdate = (deltaTime: number) => {
            if (!this.sprite || !this.sprite.isValid || this.isDestroyed) {
                this.isPlayingRepairAnimation = false;
                this.unschedule(animationUpdate);
                return;
            }

            animationTimer += deltaTime;

            if (animationTimer >= this.repairAnimationDuration) {
                // 动画播放完成，恢复默认精灵
                if (this.defaultSpriteFrame) {
                    this.sprite.spriteFrame = this.defaultSpriteFrame;
                }
                this.isPlayingRepairAnimation = false;
                this.unschedule(animationUpdate);
                return;
            }

            const    targetFrameIndex    =    Math.min(Math.floor(animationTimer    /
frameDuration), frameCount - 1);
            if (targetFrameIndex !== lastFrameIndex && targetFrameIndex < frameCount &&
frames[targetFrameIndex]) {
                this.sprite.spriteFrame = frames[targetFrameIndex];
                lastFrameIndex = targetFrameIndex;
            }
        };

        this.unschedule(animationUpdate);
        this.schedule(animationUpdate, 0);
    }

    /**
     * 移动到维修目标附近
     */
    private moveToRepairTarget(target: Node, deltaTime: number) {
        if (!target || !target.isValid) {
            return;
        }

        const currentPos = this.node.worldPosition;
```

```
        const targetPos = target.worldPosition;

        // 计算方向
        const direction = new Vec3();
        Vec3.subtract(direction, targetPos, currentPos);
        direction.normalize();

        // 计算新位置
        const moveDistance = this.moveSpeed * deltaTime;
        const newPos = new Vec3();
        Vec3.scaleAndAdd(newPos, currentPos, direction, moveDistance);

        // 更新位置
        this.node.setWorldPosition(newPos);
        // 不设置 isMoving = true，避免影响手动控制判断

        // 根据移动方向翻转精灵
        if (direction.x < 0) {
            this.node.setScale(-Math.abs(this.defaultScale.x),   this.defaultScale.y,
this.defaultScale.z);
        } else {
            this.node.setScale(Math.abs(this.defaultScale.x),    this.defaultScale.y,
this.defaultScale.z);
        }
    }

    /**
     * 获取建筑物脚本
     */
    private getBuildingScript(building: Node): any {
        const scriptNames = [
            'WarAncientTree',
            'HunterHall',
            'StoneWall',
            'WatchTower',
            'IceTower',
            'ThunderTower',
            'SwordsmanHall',
            'Church',
            'MoonWell'
        ];

        for (const scriptName of scriptNames) {
            const script = building.getComponent(scriptName as any);
            if (script) {
                return script;
            }
        }

        return null;
    }

    /**
     * 根据路径查找节点
     */
    private findNodeByPath(path: string): Node | null {
        const parts = path.split('/');
        let current: Node | null = null;

        // 先尝试直接查找
        current = find(path);
        if (current) return current;
```

```
        // 如果直接查找失败，尝试递归查找
        const findNodeRecursive = (node: Node, name: string): Node | null => {
            if (node.name === name) {
                return node;
            }
            for (const child of node.children) {
                const found = findNodeRecursive(child, name);
                if (found) return found;
            }
            return null;
        };

        // 从场景根节点开始查找最后一个部分
        if (this.node.scene && parts.length > 0) {
            const targetName = parts[parts.length - 1];
            return findNodeRecursive(this.node.scene, targetName);
        }

        return null;
    }

    /**
     * 显示维修特效
     */
    private showRepairEffect(target: Node) {
        if (this.healEffectPrefab && target && target.isValid) {
            const effect = instantiate(this.healEffectPrefab);
            const canvas = find('Canvas');
            if (canvas) {
                effect.setParent(canvas);
            } else if (this.node.scene) {
                effect.setParent(this.node.scene);
            }
            effect.setWorldPosition(target.worldPosition.clone().add3f(0, 20, 0));
            effect.active = true;

            // 延迟销毁
            this.scheduleOnce(() => {
                if (effect && effect.isValid) {
                    effect.destroy();
                }
            }, 1.0);
        }
    }

    /**
     * 显示维修数字
     */
    private showRepairNumber(target: Node, amount: number) {
        if (this.damageNumberPrefab && target && target.isValid) {
            const healNode = instantiate(this.damageNumberPrefab);
            const canvas = find('Canvas');
            if (canvas) {
                healNode.setParent(canvas);
            } else if (this.node.scene) {
                healNode.setParent(this.node.scene);
            }

            // 设置治疗数字位置（在建筑物上方）
            const targetPos = target.worldPosition.clone();
            healNode.setWorldPosition(targetPos.add3f(0, 50, 0));
            healNode.active = true;
```

```
            // 获取 DamageNumber 组件并设置治疗数值和颜色
            const healScript = healNode.getComponent('DamageNumber' as any) as any;
            if (healScript) {
                // 设置治疗量，使用负数表示治疗
                healScript.setDamage(-amount);
                // 设置为绿色
                healScript.setColor(new Color(0, 255, 0, 255));
            }
        }
    }

    /**
     * 更新小精灵在树林网格中的半透明隐身效果：
     * - 进入任意一片树林网格：身体变淡到 50% 不透明度
     * - 离开所有树林网格：恢复原始不透明度
     */
    private updateForestStealthState() {
        if (!this.sprite || !this.node || !this.node.isValid) {
            return;
        }

        const worldPos = this.node.worldPosition.clone();
        let inForest = false;

        // 左侧树林
        const leftNode = find('Canvas/ForestGridLeft') || find('ForestGridLeft');
        if (leftNode) {
            const panel = leftNode.getComponent(ForestGridPanel);
            if (panel && panel.worldToGrid(worldPos)) {
                inForest = true;
            }
        }

        // 右侧树林（如果左侧未命中）
        if (!inForest) {
            const      rightNode      =      find('Canvas/ForestGridRight')      ||
find('ForestGridRight');
            if (rightNode) {
                const panel = rightNode.getComponent(ForestGridPanel);
                if (panel && panel.worldToGrid(worldPos)) {
                    inForest = true;
                }
            }
        }

        if (inForest === this.isInForestArea) {
            // 状态未变化，无需重复设置
            return;
        }

        this.isInForestArea = inForest;

        if (!this.originalSpriteColor) {
            this.originalSpriteColor = this.sprite.color.clone();
        }

        if (this.isInForestArea) {
            // 进入树林：将透明度降到 50%
            const c = this.originalSpriteColor;
            this.sprite.color = new Color(c.r, c.g, c.b, 128);
        } else {
            // 离开树林：恢复原有颜色（包含原始透明度）
            this.sprite.color = this.originalSpriteColor.clone();
```

```
        }
    }

    /**
     * 查找游戏管理器
     */
    findGameManager() {
        let gmNode = find('GameManager');
        if (gmNode) {
            this.gameManager = gmNode.getComponent(GameManager);
            if (this.gameManager) {
                return;
            }
        }

        const scene = this.node.scene;
        if (scene) {
            const findInScene = (node: Node, componentType: any): any => {
                const comp = node.getComponent(componentType);
                if (comp) return comp;
                for (const child of node.children) {
                    const found = findInScene(child, componentType);
                    if (found) return found;
                }
                return null;
            };
            this.gameManager = findInScene(scene, GameManager);
        }
    }

    /**
     * 查找单位选择管理器
     */
    findUnitSelectionManager() {
        let managerNode = find('UnitSelectionManager');
        if (managerNode) {
            this.unitSelectionManager                                                    =
managerNode.getComponent(UnitSelectionManager);
            if (this.unitSelectionManager) {
                return;
            }
        }

        const scene = this.node.scene;
        if (scene) {
            const findInScene = (node: Node, componentType: any): any => {
                const comp = node.getComponent(componentType);
                if (comp) return comp;
                for (const child of node.children) {
                    const found = findInScene(child, componentType);
                    if (found) return found;
                }
                return null;
            };
            this.unitSelectionManager = findInScene(scene, UnitSelectionManager);
        }
    }

    /**
     * 依附到建筑物
     * @param building 要依附的建筑物
     * @param fromBuilding 是否由建筑物主动调用（用于避免循环调用）
     */
```

```
    attachToBuilding(building: Node, fromBuilding: boolean = false) {
        if (this.isAttached) {
            console.warn('Wisp: Already attached to a building!');
            return;
        }

        // 保存建筑物引用，但不立即设置依附状态
        this.attachedBuilding = building;
        this.isMoving = false;
        this.moveTarget = null!;

        // 立即更新位置
        const buildingPos = building.worldPosition.clone();
        const attachPos = buildingPos.add(this.attachOffset);
        this.node.setWorldPosition(attachPos);

        // 不要隐藏小精灵节点，否则 update 方法不会被调用，无法执行治疗逻辑
        // 保持节点 active 为 true，让 update 方法继续执行治疗逻辑
        // this.node.active = false;

        // 如果是小精灵主动依附（不是由建筑物调用），需要通知建筑物添加到依附列表
        if (!fromBuilding) {
            const buildingScript = building.getComponent('WarAncientTree') as any;
            if (buildingScript && buildingScript.attachWisp) {
                buildingScript.attachWisp(this.node);
            } else {
                const moonWellScript = building.getComponent('MoonWell') as any;
                if (moonWellScript && moonWellScript.attachWisp) {
                    moonWellScript.attachWisp(this.node);
                }
            }
        }

        // 最后设置依附状态
        this.isAttached = true;

        console.debug('Wisp: Attached to building and disappeared into it');
    }

    /**
     * 从建筑物卸下
     */
    detachFromBuilding() {
        if (!this.isAttached) {
            return;
        }

        const building = this.attachedBuilding;
        this.attachedBuilding = null!;
        this.isAttached = false;
        this.isMoving = true; // 设置为移动状态，避免立即重新依附
        this.moveTarget = null!;

        // 小精灵重新显示
        this.node.active = true;

        // 设置位置到建筑物下方更远的位置，超出依附范围
        if (building && building.isValid) {
            const buildingPos = building.worldPosition.clone();
            // 建筑物前方位置（下方 100 像素，超出 50 像素的依附范围）
            const frontPos = new Vec3(buildingPos.x, buildingPos.y - 100, buildingPos.z);
            this.node.setWorldPosition(frontPos);
        }
```

```
        // 延迟重置移动状态，确保不会立即重新依附
        this.scheduleOnce(() => {
            this.isMoving = false;
        }, 1.0);

        console.debug('Wisp: Detached from building and reappeared');
    }

    /**
     * 设置手动移动目标位置（用于选中移动）
     * 覆盖父类实现：清除维修目标
     * @param worldPos 世界坐标位置
     */
    setManualMoveTargetPosition(worldPos: Vec3) {
        // 清除维修目标
        this.repairTarget = null!;
        this.isRepairing = false;

        // 调用父类的手动移动逻辑（带智能分散/避让）
        super.setManualMoveTargetPosition(worldPos);

        console.debug(`Wisp: Manual move target set to (${worldPos.x.toFixed(1)},
${worldPos.y.toFixed(1)})`);
    }

    /**
     * 检查目标位置附近是否有建筑物，如果有则依附
     * @param targetPos 目标位置
     * @returns 是否找到并依附到建筑物
     */
    private checkBuildingNearbyAndAttach(targetPos: Vec3): boolean {
        const attachRange = 10; // 依附范围（像素），当距离建筑物 10 像素以内时依附

        // 查找战争古树
        let treesNode = find('WarAncientTrees');
        if (!treesNode && this.node.scene) {
            const findNodeRecursive = (node: Node, name: string): Node | null => {
                if (node.name === name) {
                    return node;
                }
                for (const child of node.children) {
                    const found = findNodeRecursive(child, name);
                    if (found) return found;
                }
                return null;
            };
            treesNode = findNodeRecursive(this.node.scene, 'WarAncientTrees');
        }

        if (treesNode) {
            const trees = treesNode.children || [];
            for (const tree of trees) {
                if (tree && tree.isValid && tree.active) {
                    const treeScript = tree.getComponent('WarAncientTree') as any;
                    if (treeScript && treeScript.isAlive && treeScript.isAlive()) {
                        const distance = Vec3.distance(targetPos, tree.worldPosition);
                        // 如果距离建筑物很近（10 像素以内），立即依附
                        if (distance <= attachRange) {
                            // 依附到建筑物
                            this.attachToBuilding(tree);
                            return true;
                        }
```

```
                }
            }
        }
    }

    // 查找月亮井
    let wellsNode = find('MoonWells');
    if (!wellsNode && this.node.scene) {
        const findNodeRecursive = (node: Node, name: string): Node | null => {
            if (node.name === name) {
                return node;
            }
            for (const child of node.children) {
                const found = findNodeRecursive(child, name);
                if (found) return found;
            }
            return null;
        };
        wellsNode = findNodeRecursive(this.node.scene, 'MoonWells');
    }

    if (wellsNode) {
        const wells = wellsNode.children || [];
        for (const well of wells) {
            if (well && well.isValid && well.active) {
                const wellScript = well.getComponent('MoonWell') as any;
                if (wellScript && wellScript.isAlive && wellScript.isAlive()) {
                    const distance = Vec3.distance(targetPos, well.worldPosition);
                    // 如果距离建筑物很近（10 像素以内），立即依附
                    if (distance <= attachRange) {
                        // 依附到建筑物
                        this.attachToBuilding(well);
                        return true;
                    }
                }
            }
        }
    }

    return false;
}

/**
 * 检查是否与建筑物重叠，如果重叠则自动依附
 */
checkBuildingOverlap() {
    // 如果已经依附，不需要检查
    if (this.isAttached) {
        return;
    }

    // 如果正在维修，不检查重叠（避免维修时被依附）
    if (this.isRepairing) {
        return;
    }

    // 如果正在移动，不检查重叠（避免移动时被依附）
    if (this.isMoving || this.isManuallyControlled) {
        return;
    }

    const wispPos = this.node.worldPosition;
    const attachRange = 50; // 依附范围（像素）
```

```
        // 查找战争古树
        const findNodeRecursive = (node: Node, name: string): Node | null => {
            if (node.name === name) {
                return node;
            }
            for (const child of node.children) {
                const found = findNodeRecursive(child, name);
                if (found) return found;
            }
            return null;
        };

        let treesNode = find('WarAncientTrees');
        if (!treesNode && this.node.scene) {
            treesNode = findNodeRecursive(this.node.scene, 'WarAncientTrees');
        }
        if (treesNode) {
            const trees = treesNode.children || [];
            for (const tree of trees) {
                if (tree && tree.isValid && tree.active) {
                    const treeScript = tree.getComponent('WarAncientTree') as any;
                    if (treeScript && treeScript.isAlive && treeScript.isAlive()) {
                        const distance = Vec3.distance(wispPos, tree.worldPosition);
                        if (distance <= attachRange) {
                            // 与战争古树重叠，自动依附
                            this.attachToBuilding(tree);
                            return;
                        }
                    }
                }
            }
        }

        // 查找月亮井
        let wellsNode = find('MoonWells');
        if (!wellsNode && this.node.scene) {
            wellsNode = findNodeRecursive(this.node.scene, 'MoonWells');
        }
        if (wellsNode) {
            const wells = wellsNode.children || [];
            for (const well of wells) {
                if (well && well.isValid && well.active) {
                    const wellScript = well.getComponent('MoonWell') as any;
                    if (wellScript && wellScript.isAlive && wellScript.isAlive()) {
                        const distance = Vec3.distance(wispPos, well.worldPosition);
                        if (distance <= attachRange) {
                            // 与月亮井重叠，自动依附
                            this.attachToBuilding(well);
                            return;
                        }
                    }
                }
            }
        }
    }

    /**
     * 治疗建筑物
     */
    healBuilding() {
        if (!this.attachedBuilding || !this.attachedBuilding.isValid) {
            return;
```

```
        }

        // 根据建筑物类型调用相应的治疗/恢复方法
        const buildingScript = this.attachedBuilding.getComponent('WarAncientTree') as
any;
        if (buildingScript && buildingScript.isAlive && buildingScript.isAlive()) {
            // 检查建筑物是否满血，避免不必要的治疗
            if (buildingScript.getHealth && buildingScript.getMaxHealth) {
                const currentHealth = buildingScript.getHealth();
                const maxHealth = buildingScript.getMaxHealth();
                if (currentHealth >= maxHealth) {
                    return; // 满血，不治疗
                }
            }

            let healed = false;
            if (buildingScript.heal) {
                // 保存治疗前的血量
                const       beforeHealth       =       buildingScript.getHealth       ?
buildingScript.getHealth() : buildingScript.currentHealth;
                buildingScript.heal(this.healAmount);
                // 检查治疗后是否真的恢复了血量
                const       afterHealth       =       buildingScript.getHealth       ?
buildingScript.getHealth() : buildingScript.currentHealth;
                healed = afterHealth > beforeHealth;
            } else if (buildingScript.getHealth && buildingScript.getMaxHealth) {
                const currentHealth = buildingScript.getHealth();
                const maxHealth = buildingScript.getMaxHealth();
                if (currentHealth < maxHealth) {
                    // 直接设置血量
                    if (buildingScript.currentHealth !== undefined) {
                        buildingScript.currentHealth       =       Math.min(maxHealth,
currentHealth + this.healAmount);
                        if (buildingScript.healthBar) {

buildingScript.healthBar.setHealth(buildingScript.currentHealth);
                        }
                        healed = true;
                    }
                }
            }
            if (healed) {
                this.showHealEffect();
                this.showHealNumber();
            }
            return;
        }

        const moonWellScript = this.attachedBuilding.getComponent('MoonWell') as any;
        if (moonWellScript && moonWellScript.isAlive && moonWellScript.isAlive()) {
            // 检查建筑物是否满血，避免不必要的治疗
            if (moonWellScript.getHealth && moonWellScript.getMaxHealth) {
                const currentHealth = moonWellScript.getHealth();
                const maxHealth = moonWellScript.getMaxHealth();
                if (currentHealth >= maxHealth) {
                    return; // 满血，不治疗
                }
            }

            let healed = false;
            if (moonWellScript.heal) {
                // 保存治疗前的血量
                const       beforeHealth       =       moonWellScript.getHealth       ?
```

```
moonWellScript.getHealth() : moonWellScript.currentHealth;
                moonWellScript.heal(this.healAmount);
                // 检查治疗后是否真的恢复了血量
                const       afterHealth       =       moonWellScript.getHealth       ?
moonWellScript.getHealth() : moonWellScript.currentHealth;
                healed = afterHealth > beforeHealth;
            } else if (moonWellScript.getHealth && moonWellScript.getMaxHealth) {
                const currentHealth = moonWellScript.getHealth();
                const maxHealth = moonWellScript.getMaxHealth();
                if (currentHealth < maxHealth) {
                    if (moonWellScript.currentHealth !== undefined) {
                        moonWellScript.currentHealth       =       Math.min(maxHealth,
currentHealth + this.healAmount);
                        if (moonWellScript.healthBar) {

moonWellScript.healthBar.setHealth(moonWellScript.currentHealth);
                        }
                        healed = true;
                    }
                }
            }
            if (healed) {
                this.showHealEffect();
                this.showHealNumber();
            }
            return;
        }
    }

    /**
     * 显示治疗数字
     */
    private showHealNumber() {
        if      (this.damageNumberPrefab      &&      this.attachedBuilding      &&
this.attachedBuilding.isValid) {
            const healNode = instantiate(this.damageNumberPrefab);
            const canvas = find('Canvas');
            if (canvas) {
                healNode.setParent(canvas);
            } else if (this.node.scene) {
                healNode.setParent(this.node.scene);
            }

            // 设置治疗数字位置（在建筑物上方）
            const buildingPos = this.attachedBuilding.worldPosition.clone();
            healNode.setWorldPosition(buildingPos.add3f(0, 50, 0));

            // 获取 DamageNumber 组件并设置治疗数值和颜色
            const healScript = healNode.getComponent('DamageNumber') as any;
            if (healScript) {
                // 设置治疗量，使用正数表示治疗
                healScript.setDamage(-this.healAmount);
                // 设置为绿色
                healScript.setColor(new Color(0, 255, 0, 255));
            }
        }
    }

    /**
     * 显示治疗特效
     */
    showHealEffect() {
        if (this.healEffectPrefab) {
```

```
            const effect = instantiate(this.healEffectPrefab);
            const canvas = find('Canvas');
            if (canvas) {
                effect.setParent(canvas);
            } else if (this.node.scene) {
                effect.setParent(this.node.scene);
            }

effect.setWorldPosition(this.attachedBuilding.worldPosition.clone().add3f(0, 20, 0));
            effect.active = true;

            // 延迟销毁
            this.scheduleOnce(() => {
                if (effect && effect.isValid) {
                    effect.destroy();
                }
            }, 1.0);
        }
    }

    /**
     * 覆盖父类的索敌逻辑：小精灵不索敌、不攻击敌人
     */
    findTarget() {
        this.currentTarget = null!;
    }

    /**
     * 覆盖父类的攻击逻辑：小精灵不能攻击敌人
     */
    attack() {
        // 小精灵是辅助单位，没有攻击能力，这里什么都不做
    }

    /**
     * 覆盖父类的实际攻击执行逻辑，确保不会发射子弹/箭矢
     */
    executeAttack() {
        // 空实现，避免父类创建子弹或箭矢
    }

    /**
     * 受到伤害
     */
    takeDamage(damage: number) {
        if (this.isDestroyed) {
            return;
        }

        // 依附在建筑物上时不会被攻击
        if (this.isAttached) {
            console.debug('Wisp.takeDamage: Wisp is attached to building, ignoring
damage');
            return;
        }

        this.currentHealth -= damage;

        // 更新单位信息面板
        if (this.unitSelectionManager &&
this.unitSelectionManager.isUnitSelected(this.node)) {
            this.unitSelectionManager.updateUnitInfo({
                currentHealth: Math.max(0, this.currentHealth),
```

```
                maxHealth: this.maxHealth
            });
        }

        // 显示伤害数字
        if (this.damageNumberPrefab) {
            const damageNode = instantiate(this.damageNumberPrefab);
            const canvas = find('Canvas');
            if (canvas) {
                damageNode.setParent(canvas);
            } else if (this.node.scene) {
                damageNode.setParent(this.node.scene);
            }
            damageNode.setWorldPosition(this.node.worldPosition.clone().add3f(0,  30,
0));
            const damageScript = damageNode.getComponent(DamageNumber);
            if (damageScript) {
                damageScript.setDamage(damage);
            }
        }

        // 更新血条
        if (this.healthBar) {
            this.healthBar.setHealth(this.currentHealth);
        }

        if (this.currentHealth <= 0) {
            this.die();
        }
    }

    /**
     * 恢复血量
     * @param amount 恢复的血量
     */
    heal(amount: number) {
        if (this.isDestroyed) {
            return;
        }

        // 如果满血，不恢复
        if (this.currentHealth >= this.maxHealth) {
            return;
        }

        // 保存治疗前的血量
        const oldHealth = this.currentHealth;
        // 恢复血量，不超过最大值
        this.currentHealth = Math.min(this.currentHealth + amount, this.maxHealth);

        // 更新血条
        if (this.healthBar) {
            this.healthBar.setHealth(this.currentHealth);
        }

        // 更新单位信息面板
        if                        (this.unitSelectionManager                        &&
this.unitSelectionManager.isUnitSelected(this.node)) {
            this.unitSelectionManager.updateUnitInfo({
                currentHealth: this.currentHealth,
                maxHealth: this.maxHealth
            });
        }
```

```
        // 显示治疗数字
        if (this.damageNumberPrefab) {
            const healNode = instantiate(this.damageNumberPrefab);
            const canvas = find('Canvas');
            if (canvas) {
                healNode.setParent(canvas);
            } else if (this.node.scene) {
                healNode.setParent(this.node.scene);
            }
            healNode.setWorldPosition(this.node.worldPosition.clone().add3f(0,    30,
0));
            const healScript = healNode.getComponent(DamageNumber);
            if (healScript) {
                // 使用负数表示治疗
                healScript.setDamage(-amount);
                // 设置为绿色
                healScript.setColor(new Color(0, 255, 0, 255));
            }
        }
    }

    /**
     * 死亡
     */
    die() {
        if (this.isDestroyed) {
            return;
        }

        this.isDestroyed = true;

        // 播放爆炸特效
        if (this.explosionEffect) {
            const explosion = instantiate(this.explosionEffect);
            const canvas = find('Canvas');
            if (canvas) {
                explosion.setParent(canvas);
            } else if (this.node.scene) {
                explosion.setParent(this.node.scene);
            }
            explosion.setWorldPosition(this.node.worldPosition);
            explosion.active = true;
        }

        // 销毁节点
        this.scheduleOnce(() => {
            if (this.node && this.node.isValid) {
                this.node.destroy();
            }
        }, 0.5);
    }

    /**
     * 是否存活
     */
    isAlive(): boolean {
        return !this.isDestroyed && this.currentHealth > 0;
    }
```

```
    /**
     * 获取血量
     */
    getHealth(): number {
        return this.currentHealth;
    }

    /**
     * 获取最大血量
     */
    getMaxHealth(): number {
        return this.maxHealth;
    }

    /**
     * 是否已依附
     */
    getIsAttached(): boolean {
        return this.isAttached;
    }

    /**
     * 获取依附的建筑物
     */
    getAttachedBuilding(): Node | null {
        return this.attachedBuilding;
    }

    /**
     * 重写碰撞检测方法：小精灵不受任何障碍物阻挡
     * @param position 要检测的位置
     * @returns 始终返回 false，表示没有碰撞
     */
    checkCollisionAtPosition(position: Vec3): boolean {
        // 小精灵可以穿过所有障碍物，不进行碰撞检测
        return false;
    }

    // 选择和高亮逻辑由父类 Role 和 UnitSelectionManager 统一管理
}
```